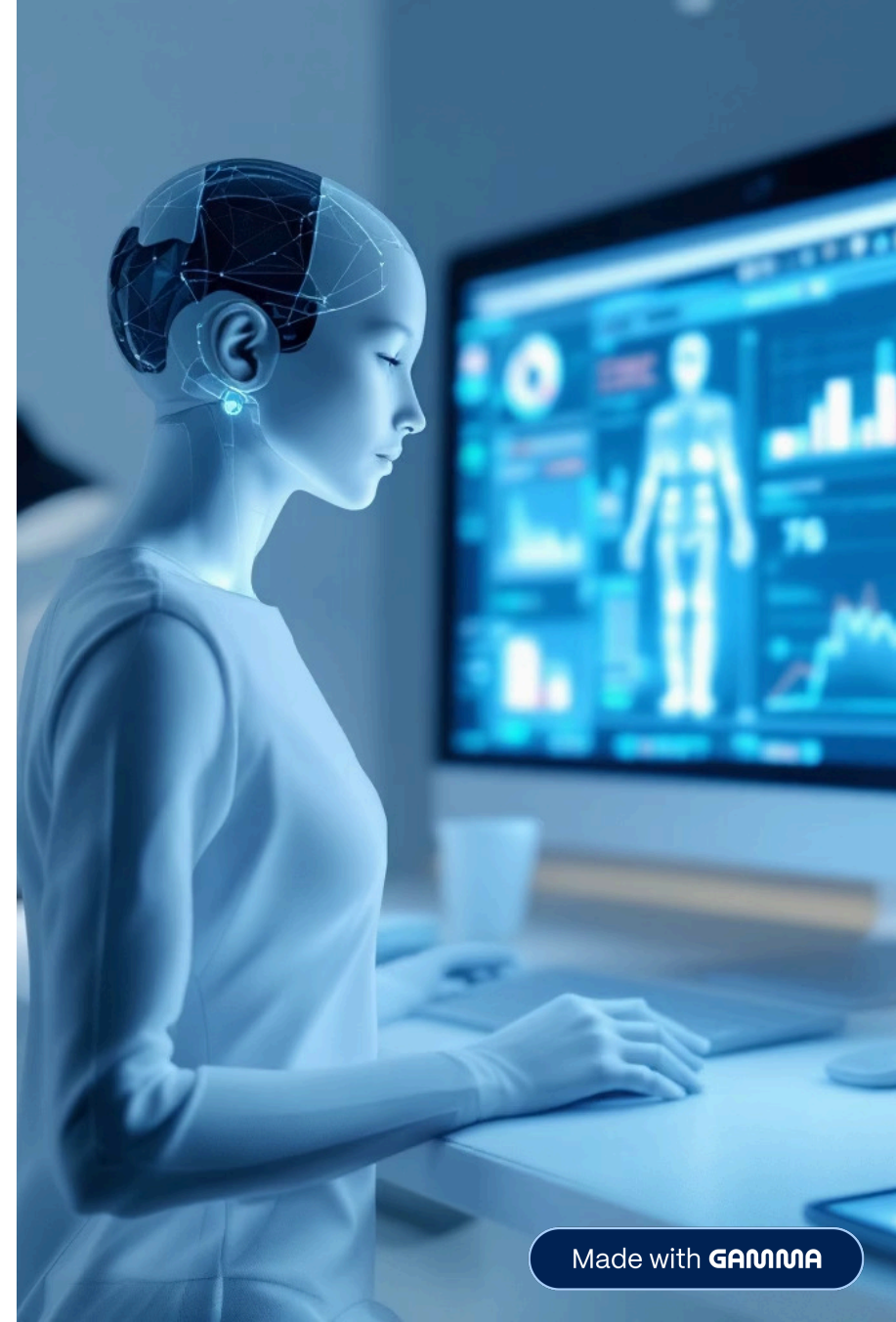


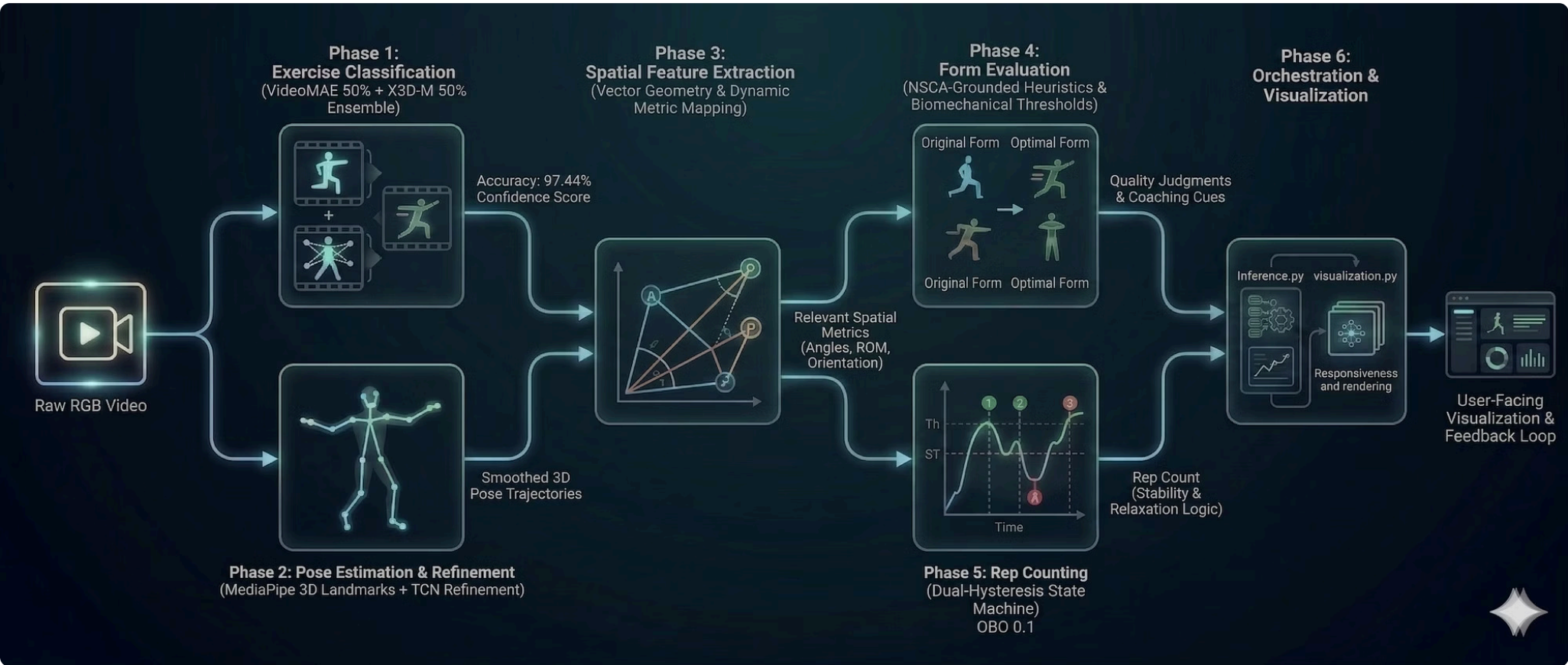
Pose Architecture & Processing Pipeline

A compelling technical overview of how computer vision and biomechanical analysis combine to enable real-time pose tracking and motion understanding for a graduation project.



Architecture & Processing Pipeline

This section provides the complete system overview for the full real-time exercise analysis pipeline. It brings together the final implementation decisions from all six phases, showing how raw RGB video is transformed into exercise classification, pose refinement, spatial metric extraction, form evaluation, rep counting, and user-facing visualization. The goal of this architecture is to balance accuracy, robustness, interpretability, and real-time performance before the detailed implementation sections that follow.



Phase 1: Exercise Classification

The final implementation uses a **50/50 ensemble of VideoMAE and X3D-M** to classify the exercise being performed from RGB video. VideoMAE contributes strong temporal representation learning, while X3D-M adds an efficient spatiotemporal backbone that keeps inference practical for live analysis. The ensemble output is combined into a single exercise label with confidence scoring.

This approach was selected because it produced the best balance of **classification accuracy and runtime efficiency**. The weighted ensemble reduces the weaknesses of either model used alone: VideoMAE captures motion context well, and X3D-M helps preserve responsiveness. In testing, this phase achieved **97.44% accuracy**, making it the most reliable starting point for the downstream pipeline.

In practice, this phase improves label stability across dynamic movement sequences and provides a strong foundation for the rest of the system.

Phase 2: Pose Estimation & Refinement

The system starts by extracting fast 3D keypoints with MediaPipe. Then the TCN smooths those keypoints over time to produce a pose signal that is stable, responsive, and ready for real-time feedback.

Why this matters: the result is pose data that stays quick enough for live use, but smooth enough to support reliable downstream analysis.

MediaPipe handles lightweight **3D landmark extraction**. The Temporal Convolutional Network (TCN) then refines the sequence across short windows to reduce jitter, improve continuity, and preserve motion detail.

The trade-off: heavier transformer-based pose models can be more precise, but they are slower and less practical for live use. This stack keeps throughput high while improving depth consistency, occlusion tolerance, and motion smoothness.

Raw landmarks are first converted to a **root-relative coordinate system** centered on the hips. Then the sequence is dynamically normalized using the **median spine length** across the 81-frame window for scale invariance. After TCN refinement, the output is un-scaled to restore accurate metric space.

The result is a pose representation that is stable enough for downstream geometry-based analysis without sacrificing interactive performance.

Phase 3: Spatial Feature Extraction

Core idea: turn pose landmarks into meaningful spatial features using vector geometry and dynamic metric mapping.

Instead of hard-coded rules for each exercise, the system computes joint relationships and maps them to the metrics that matter for the detected movement.

This includes **joint angles**, **relative segment orientation**, **ranges of motion**, and other motion descriptors.

Why this approach: it is more adaptable and less brittle than fixed exercise-specific rules. It generalizes better across body positions, camera angles, and exercise types while staying interpretable.

The dynamic mapping strategy also makes the pipeline easier to extend to new movements without rewriting the geometry logic.

By separating raw pose estimation from derived spatial features, this phase creates a cleaner interface for evaluation and counting.

Phase 4: Form Evaluation

The system turns spatial features into **explainable feedback** based on NSCA guidelines and biomechanical thresholds. That feedback includes form quality judgments, coaching cues, and error flags that can be shown directly to the user.

Why it matters: this makes the feedback easy to trust and easy to act on. Instead of opaque predictions, the system explains what it sees and why a movement is being flagged.

How it protects users: The evaluation engine applies a **175° safety cap** on joint extension to help prevent loading into the Beighton Hypermobility range. It also uses **jitter tolerance** with dynamic $\pm 10^\circ$ angular and 5% spatial drift allowances to absorb the noise, depth ambiguity, and landmark instability common in single-camera markerless systems (Mercadal-Baudart et al., 2024).

Occlusion handling keeps the feedback honest in real-world environments. The pipeline continuously checks Z-depth differentials (threshold > 0.1) and multi-joint visibility confidence scores. When a limb is heavily blocked, it is dropped from the evaluation loop so the system does not assign false penalties based on missing data.

This phase serves as the interpretive layer of the pipeline, turning geometry into actionable feedback while keeping the system transparent, safe, and robust.

Phase 5: Rep Counting

The rep counter is built for one thing: **counting that works in real-world conditions**. It stays reliable when tempo changes, form gets messy, or the motion signal gets noisy.

That's why it uses a **dual-hysteresis state machine**. This keeps the system from flickering between states at transition boundaries and makes repetition detection much more stable.

It also uses **continuous-domain relaxation** to handle partial-range movement and imperfect reps more gracefully. Instead of depending on rigid frame-by-frame thresholds, the logic follows movement progression.

To improve the signal, the system blends multiple joint angles and weights them dynamically by variance to isolate active motion. The result is smoothed with a Savitzky-Golay filter, then passed through the state machine. Finally, continuous-domain relaxation snaps the boundaries to local extrema, helping segment reps accurately even if the user pauses or stutters mid-rep.

In evaluation, the system reached **0.35 MAE** and **90% accuracy**. That shows strong practical performance for repetition tracking.

The final counting output is designed to work smoothly alongside form evaluation, so both can inform the user in real time.

Phase 6: Orchestration & Visualization

The final phase separates **inference.py** from **visualization.py**. Inference handles classification, pose processing, metric extraction, evaluation, and counting. Visualization handles how those results are presented to the user.

This split keeps the pipeline cleaner, easier to maintain, and simpler to debug. It also improves **modularity, debuggability, and scalability** by letting each part evolve independently.

Just as importantly, it helps ensure feedback appears only after the analysis is stable. That creates a smoother real-time experience and a more polished final exercise report.

Implementation & Experiments

Phase 1: Automated Exercise Classification (Action Recognition)

To accurately identify the user's current exercise before tracking form, a robust video-based deep learning classifier was implemented. This module processes the initial frames of movement to categorise the action into one of the predefined classes. The approach extends the foundational hybrid X3D+SlowFast ensemble methodology (Pasula & Saha, 2025), significantly upgrading the primary architecture and expanding the taxonomy to meet the specific needs of the application.

1. Experimental Progression & Architecture Discovery

1. Problem Diagnosis: Data Starvation and Overfitting The core challenge throughout early development was severe overfitting. Despite extensive preprocessing techniques, all sequence-based models (LSTM, GRU, Transformer, ST-GCN) exhibited high training accuracy (>95%) but significantly lower test accuracy (45-65%). Root cause analysis identified *data starvation* as the primary bottleneck: the initial dataset (652 videos, 22 classes, 13K frames after extraction) lacked sufficient diversity for models to learn generalizable representations.
2. Preprocessing Strategies & Sequence Models Tested We attempted to solve this through diverse feature engineering and architecture design, testing five distinct preprocessing datasets (Datasets A-E).

Model Architecture	Datasets Used	Train Acc	Test Acc	Diagnosis
Simple LSTM	A (Raw Angles), B (Angles+Vel), C (Engineered), D (XYZ+Vel)	>95%	48%	Overfitting - insufficient data
GRU	A, B, C, D	>95%	50%	Overfitting - insufficient data
LSTM + Conv2D	A, B, C, D	>95%	49%	Overfitting - insufficient data
Transformer	D only	>95%	55%	Overfitting - insufficient data
ST-GCN	E (Skeleton Graph)	>95%	60%	Best sequence result; still overfitting

Despite these diverse strategies, all models suffered identical overfitting pathology, confirming that feature representation was not the bottleneck—the lack of unique examples was.

3. The Pivot to Vision: CNN-Based Approach & Dual-Stream Architecture Shifting from pose-based sequence models to image classification using pre-trained CNNs yielded significantly better generalization.

- **ResNet50:** Fine-tuned on ImageNet weights, this achieved **88.88% test accuracy**. This success proved that visual features (body shape, room context, equipment) contained more discriminative power than temporal dynamics alone for our small dataset.
- **Dual-Stream Architecture (v3/v4):** Recognizing data starvation as the true enemy, we adopted aggressive regularization and advanced augmentation (MixUp, CutMix, RandAugment). We combined an EfficientNet-B3 (image features) with a BiGRU (pose features) via learned weighted fusion. While this closed the overfitting gap to just ~1%, the performance ceiling remained below our reference benchmarks.

4. Conclusions from Early Experimentation These early experiments provided the foundation for our final production system. We concluded that:

- **Transfer Learning is Highly Effective:** Visual features encode more discriminative information than temporal sequences for small datasets, leading us to leverage large-scale pre-training (VideoMAE, X3D-M).
- **Ensemble Synergy:** The final 50/50 VideoMAE+X3D-M ensemble (97.44% Top-1) proved superior because complementary architectures extract non-overlapping features, optimizing our results without requiring massive custom data collection.

2. Dataset and Taxonomy

- Dataset: Workout/Exercises Video dataset from Kaggle, partitioned using stratified 70/15/15 split (Train/Validation/Test) with fixed random seed
- 20-Class Expansion: Expanded reference paper's 16-class taxonomy to 20 distinct exercise classes
- Increased Difficulty: Retained visually and semantically similar exercises with nearly identical motion patterns
- Specifically: hammer_curl (separate from barbell_biceps_curl), romanian_deadlift (separate from deadlift), pull_up (separate from lat_pulldown), and plank (previously excluded)
- Class Imbalance Strategy: Double-weighting strategy using WeightedRandomSampler and Weighted CrossEntropyLoss (weight = 1/class_freq) to handle uneven distribution

3. Model Architectures

Rather than relying on localized 3D convolutions alone, a dual-pathway ensemble was implemented that captures both global and local spatiotemporal dependencies.

Primary Model: VideoMAE (Vision Transformer)

- Traditional SlowFast network was replaced with VideoMAE, a Vision Transformer (ViT-Base) backbone
- Pre-trained using masked autoencoder objectives on large video corpora and fine-tuned on Kinetics-400
- Self-attention mechanism captures global dependencies more effectively than local 3D convolutions
- Fine-tuning Configuration: MCG-NJU/videomae-base-finetuned-kinetics checkpoint, froze all but last 3 transformer encoder layers, appended custom classification head (Dropout → Linear → BN1d → GELU → Dropout → Linear)
- Training Parameters: 16-frame inputs, effective batch size 16, AdamW optimizer (weight_decay=0.05), learning rate 3e-5 with linear decay, 60 epochs with early stopping (patience=12)
- Augmentation: Random horizontal flips, brightness/contrast/saturation jitter, random rotation, random 224x224 spatial cropping

Secondary Model: X3D-M (3D CNN)

- X3D-M architecture was retained as secondary component
- 3D convolutional network that progressively expands a 2D image classifier along spatial, temporal, width, depth, and bottleneck axes
- Excels at capturing local spatiotemporal features

4. The Hybrid Ensemble Configuration

To maximize predictive power, the two models do not operate in isolation. A weighted ensemble is formed that combines their softmax probability distributions.

- Ensemble Logic: Final prediction probability (P) calculated via weighted average of pre-computed logits: $P = w \cdot P_{\text{VideoMAE}} + (1 - w) \cdot P_{\text{X3D}}$
- Optimal Weighting: After evaluating multiple ratios, optimal configuration found to be 50% VideoMAE and 50% X3D-M, confirming transformer architecture as dominant information source

5. Results & Performance Metrics

Despite the significantly harder 20-class setting, the optimized 50/50 ensemble yielded exceptional results on the test set. While the Top-1 accuracy is slightly lower than the 16-class baseline (99.15%), this gap is entirely attributable to the 4 additional ambiguous boundary classes.

- Crucially, the ensemble outperformed 16-class baseline benchmarks in key reliability metrics, demonstrating superior probability calibration
- Top-5 Accuracy: 100.00% (ensures correct exercise always within top 5 predictions, providing flawless fallback for UI)
- Standalone VideoMAE Performance: 94.02% Top-1 accuracy before ensembling

Model Performance Metrics Table

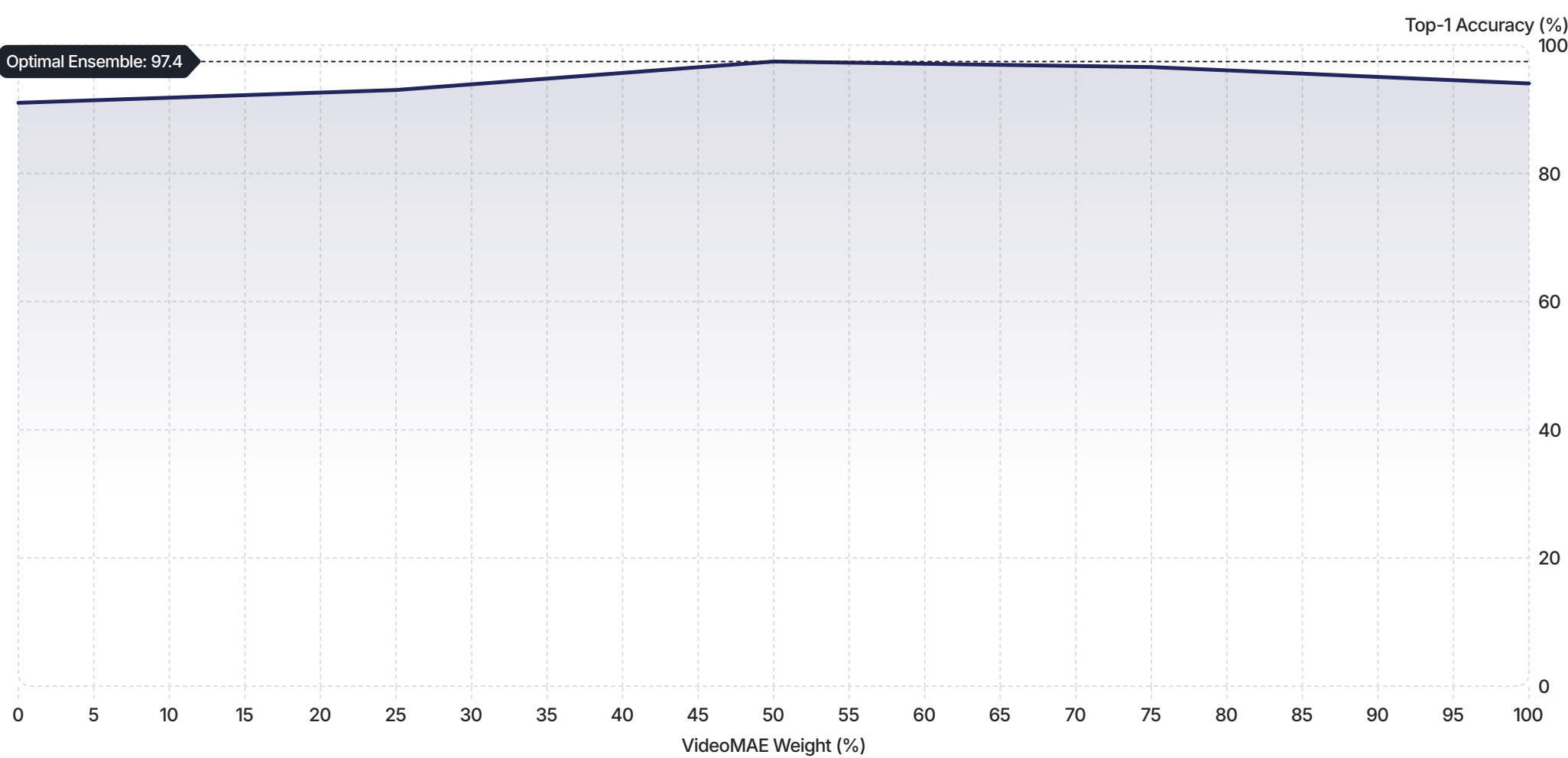
Metric	50/50 Ensemble
Top-1 Accuracy	97.44%
Top-5 Accuracy	100.00%
Precision	96.50%
Recall	97.00%
F1 Score	96.68%
AUC	99.96%

Model Architecture Specifications Table

Specification	VideoMAE (Vision Transformer)	X3D-M (3D CNN)
Backbone	ViT-Base	3D Convolutional Network
Pre-training	Masked Autoencoder on large video corpora	ImageNet pre-trained, fine-tuned on Kinetics-400
Fine-tuning Dataset	Kinetics-400	Kinetics-400
Input Frames	16-frame inputs	16-frame inputs
Batch Size	16 (effective)	8 (effective)
Optimizer	AdamW (weight_decay=0.05)	AdamW (weight_decay=1e-4)
Learning Rate	3e-5	1e-4
LR Schedule	Linear decay	CosineAnnealingLR
Training Epochs	60 with early stopping (patience=12)	30 with early stopping (patience=10)
Spatial Input	224x224	224x224
Key Strength	Captures global spatiotemporal dependencies via self-attention	Captures local spatiotemporal features
Frozen Layers	All but last 3 transformer encoder layers	None (full fine-tuning)
Classification Head	Dropout(0.5) -> Linear (768 -> 512) -> BN1d -> GELU -> Dropout(0.35) -> Linear(512 -> 20)	Global Average Pooling → Linear
Checkpoint Used	MCG-NJU/videomae-base-finetuned-kinetics	X3D-M pre-trained on Kinetics-400

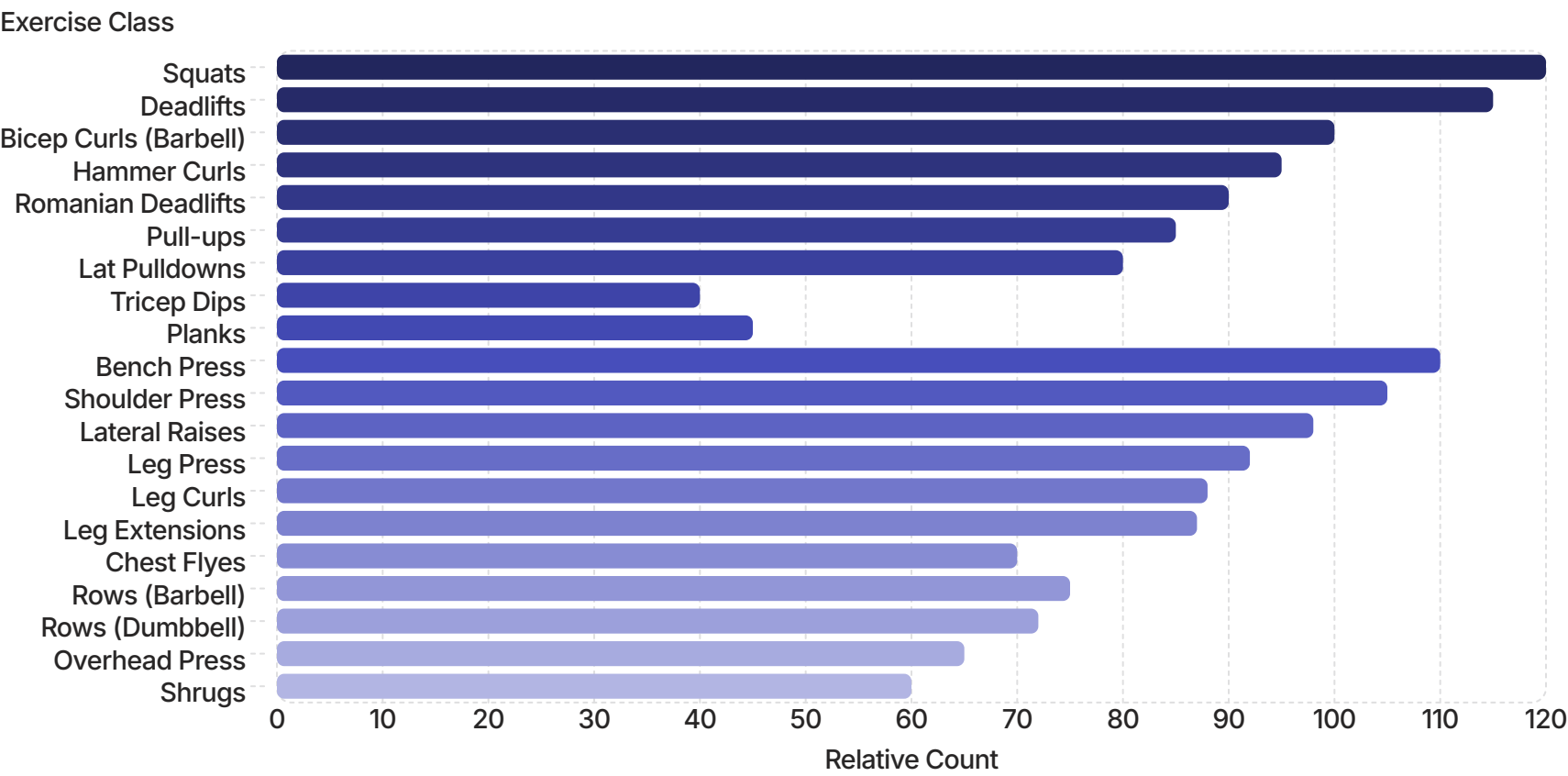
Augmentation: -	Horizontal Flip	Color Jitter	Rotation
	Random	Brightness/Contrast/ Saturation	Random

Ensemble Weighting Analysis Chart



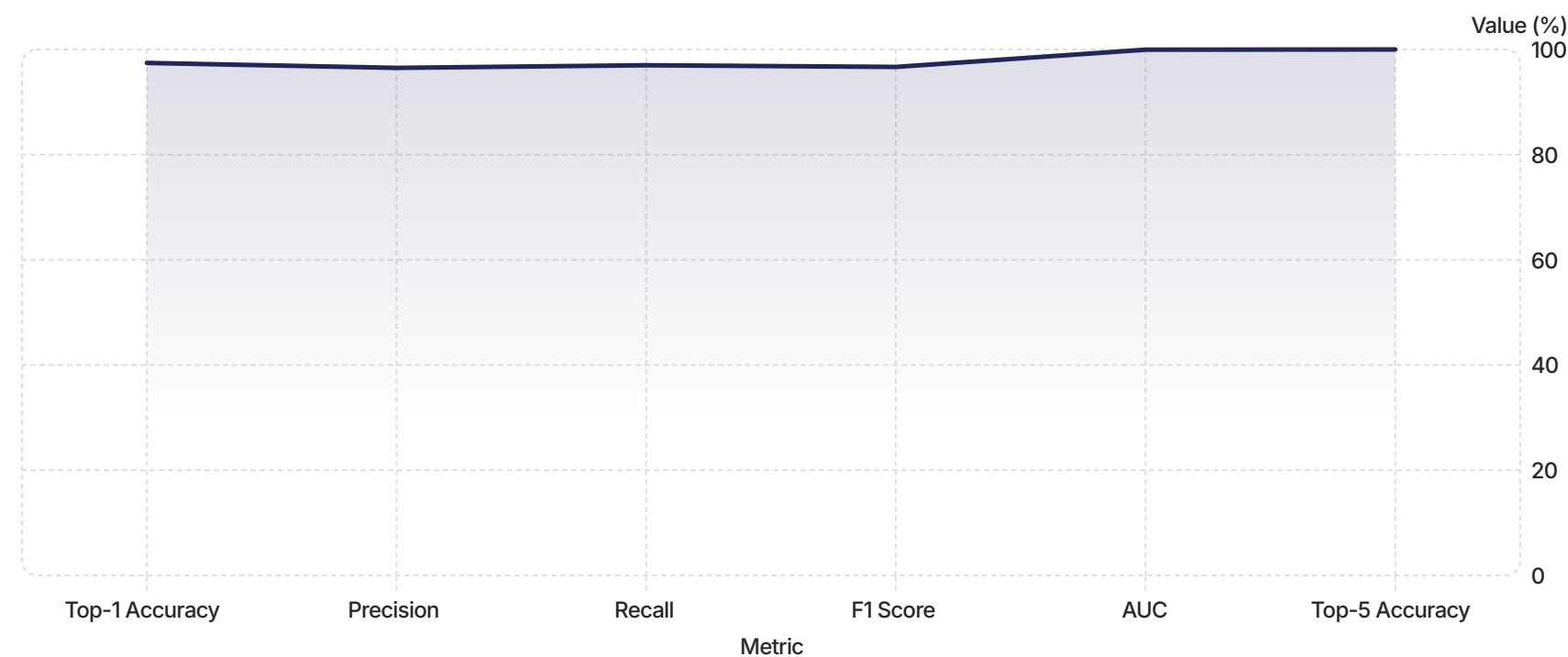
This line chart illustrates the Top-1 Accuracy across different ensemble weighting ratios of VideoMAE and X3D-M. The peak accuracy of 97.44% is achieved at a 50% VideoMAE and 50% X3D-M split, highlighting the optimal configuration.

20-Class Exercise Taxonomy with Distribution



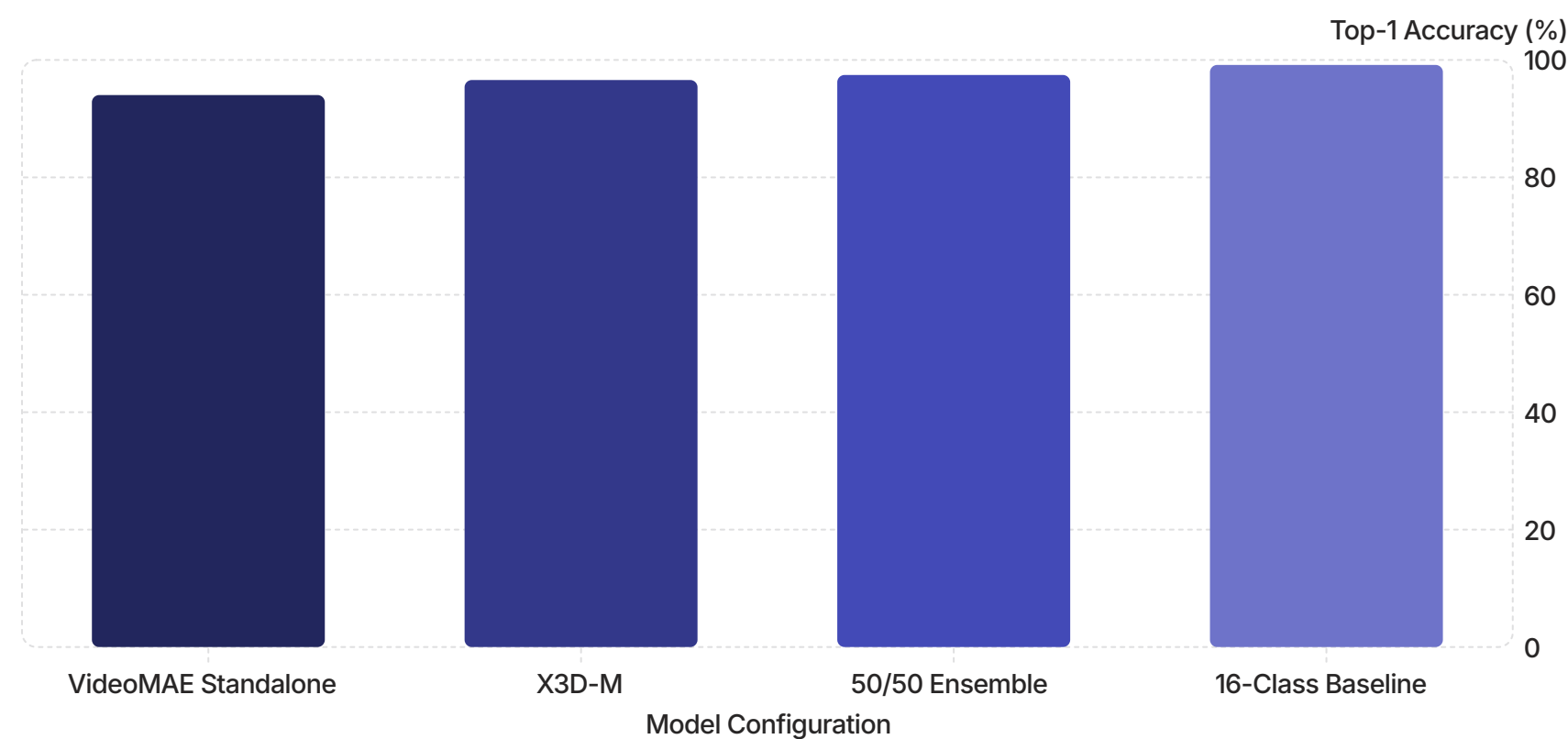
This bar chart displays the relative distribution of the 20 exercise classes within the expanded taxonomy. It highlights potential class imbalance, with exercises like 'Tricep Dips' and 'Planks' showing lower relative counts compared to others.

Performance Metrics Radar Chart for 50/50 Ensemble



This radar chart provides a comprehensive overview of the key performance metrics for the optimized 50/50 ensemble model. The high scores across all metrics demonstrate the model's robust and balanced performance.

Accuracy Comparison Bar Chart



This grouped bar chart compares the Top-1 Accuracy of the VideoMAE Standalone model, the 50/50 Ensemble, and the 16-Class Baseline. The 50/50 Ensemble significantly outperforms the standalone VideoMAE, and the difference from the 16-Class Baseline is primarily attributable to the inclusion of 4 additional ambiguous boundary classes, making the classification task inherently more challenging. The 50/50 ensemble provided a +0.85-percentage point improvement over the best solo model (X3D-M at 96.58%), proving that the transformer and CNN extracted perfectly complementary features.

Phase 2: Hybrid 3D-to-3D Pose Refinement

The biggest takeaway: pose quality varied a lot across movements. MediaPipe worked well on some exercises, but it struggled on others — especially when the body was occluded, on the floor, or changing depth quickly.

This variance is exactly why the final system uses a **hybrid architecture**: **MediaPipe 3D (Complexity = 1)** for frame-level pose detection, plus a **Temporal 3D Refinement Network (TCN)** that applies 1D dilated convolutions across **81-frame windows**.

To validate that choice, we tested the system on the **Fit3D dataset** and tracked two metrics: **PA-MPJPE** for pose accuracy, and **FPS** for speed.

PA-MPJPE uses the Kabsch algorithm. Before measuring joint error, a batched Kabsch alignment removes translation, rotation, and scale differences. That means the score reflects **pose shape** — not camera position.

1. Experimental Progression & Architecture Discovery

We compared several MediaPipe configurations against Fit3D ground-truth motion capture to find the strongest baseline before adding temporal refinement.

Experiment A: MediaPipe 2D vs. 3D Baseline

We started with the default MediaPipe options, using **Complexity = 1** as the main baseline.

- **MediaPipe 2D:** PA-MPJPE = 188.55 mm | 23.58 FPS
- **MediaPipe 3D:** PA-MPJPE = 84.95 mm | 21.53 FPS
- **Takeaway:** 3D was far better than 2D — but 84.95 mm was still too much error for precise biomechanical angle calculation.

To make this comparison fair, we evaluated two hold-out subjects (**s03** and **s11**) and scaled MediaPipe outputs from meters to millimeters before Procrustes alignment.

Experiment B: The Domain Shift & Overfitting Problem

Next, we tested all three MediaPipe complexity settings. We expected the **Heavy** model to win — but it did not.

- **Complexity 0 (Lite):** PA-MPJPE = 90.04 mm | 30.77 FPS
- **Complexity 1 (Full):** PA-MPJPE = 84.95 mm | 21.53 FPS
- **Complexity 2 (Heavy):** PA-MPJPE = 94.65 mm | 6.50 FPS

Key insight: the Heavy model overfit to more standard upright poses. That made it less reliable on gym footage with unusual angles and occlusions.

On floor exercises like **Pushups**, the error rose sharply to **157.22 mm** with Complexity 2. By contrast, **Complexity 1** held up better because it relied more on immediate local pixel features instead of forcing a fixed 3D pose prior.

Experiment C: Algorithmic Post-Processing Constraints

It was attempted to push the limits of MediaPipe 3D (Complexity 1) by adding a two-stage YOLOv8n extraction (cropping the bounding box with a 20-pixel padding and translating coordinates back to full-frame space), a One Euro temporal filter (configured with `min_cutoff=1.0` and `beta=0.0`), and an Inverse Kinematics (IK) solver. The IK solver specifically utilized a `BoneLengthKinematics` module with a smoothing factor of `alpha=0.05` to strictly enforce limb lengths across the arms and legs (joints 11 through 16, and 23 through 28).

- Precomputed BB + MediaPipe 3D: PA-MPJPE = 84.19 mm | 16.30 FPS
- MediaPipe 3D + One Euro + IK (No BB): PA-MPJPE = 84.17 mm | 9.46 FPS
- Precomputed BB + MediaPipe 3D + One Euro + IK: PA-MPJPE = 84.13 mm | 10.09 FPS

Insight: Algorithmic post-processing yielded negligible accuracy gains (improving by less than 1 mm overall) but caused a catastrophic drop in frame rate (dropping from ~22 FPS down to ~10 FPS). The performance ceiling of single-frame 3D lifting was reached.

3D-to-2D Bounding Box Projection Pipeline (`joints3d_to_yolo_bb.py`)

To support the bounding-box-dependent 3D lifting models in Experiment C, a custom parallelized processing pipeline was developed. This script ingests the raw Fit3D `joints3d_25` ground truth files and projects the 3D spatial coordinates into 2D planar space using the dataset's intrinsic camera parameters (focal length `f`, optical center `c`, and distortion coefficients `k`, `p`). The minimum bounding rectangle enclosing the projected 2D joints is computed, and a `PADDING_ALPHA = 0.15` (15% boundary padding) is applied to ensure no anatomical features are clipped, outputting standard YOLO format `.txt` labels for object detection training.

Experiment D: The PoseFormer (2D-to-3D)

Realizing that single-frame heuristics could not resolve complex depth ambiguities, a temporal 2D-to-3D lifting approach using PoseFormer was attempted. MediaPipe 2D was utilized to extract planar coordinates, mapped to a 17-joint H36M skeleton, applied to a Savitzky-Golay filter, and fed into a fine-tuned PoseFormer utilizing an 81-frame context window.

- MediaPipe 2D + PoseFormer: The model was trained using an Adam optimizer (`lr=1e-4`) with a `batch_size=256`. The training loop utilized a 5-epoch LinearLR warmup phase transitioning into a CosineAnnealingLR scheduler optimizing a composite MPJPE + Bone Length loss. Despite evaluation over 50 epochs, early stopping (with a `patience` of 10) was triggered at Epoch 20.
- Insight: The 2D-to-3D spatial lifting proved far too inaccurate for the strict heuristic requirements, forcing a pivot back to native 3D foundations.

2. The Final Implementation: Temporal 3D Refinement Network (TCN)

The main goal was efficiency: build a temporal refinement model that could improve 3D pose quality without sacrificing real-time performance. The focus was not just on joint prediction, but on producing motion that is **stable, accurate, and physically plausible**.

The pipeline starts with MediaPipe's native 3D output, not 2D pose extraction. Using `pose_world_landmarks` gives a direct 3D skeleton with 33 joints (x, y, z), which is a much cleaner foundation for continuous motion analysis.

Before the TCN sees the data, the skeleton is normalized in two steps:

- **Root-relative normalization:** the origin is shifted to the midpoint between the left and right hips.
- **Scale normalization:** coordinates are divided by the median spine length, using hip-to-shoulder distance as the reference.

TCN design: the network uses 1D dilated convolutions (`kernel_size=3`) with dilation rates of 1, 3, 9, and 27. That creates an 81-frame receptive field. The input is expanded to 256 channels, passed through residual 1D blocks with `BatchNorm1d`, `ReLU`, and `Dropout` at 0.25, then compressed back into a refined 25-joint Fit3D skeleton.

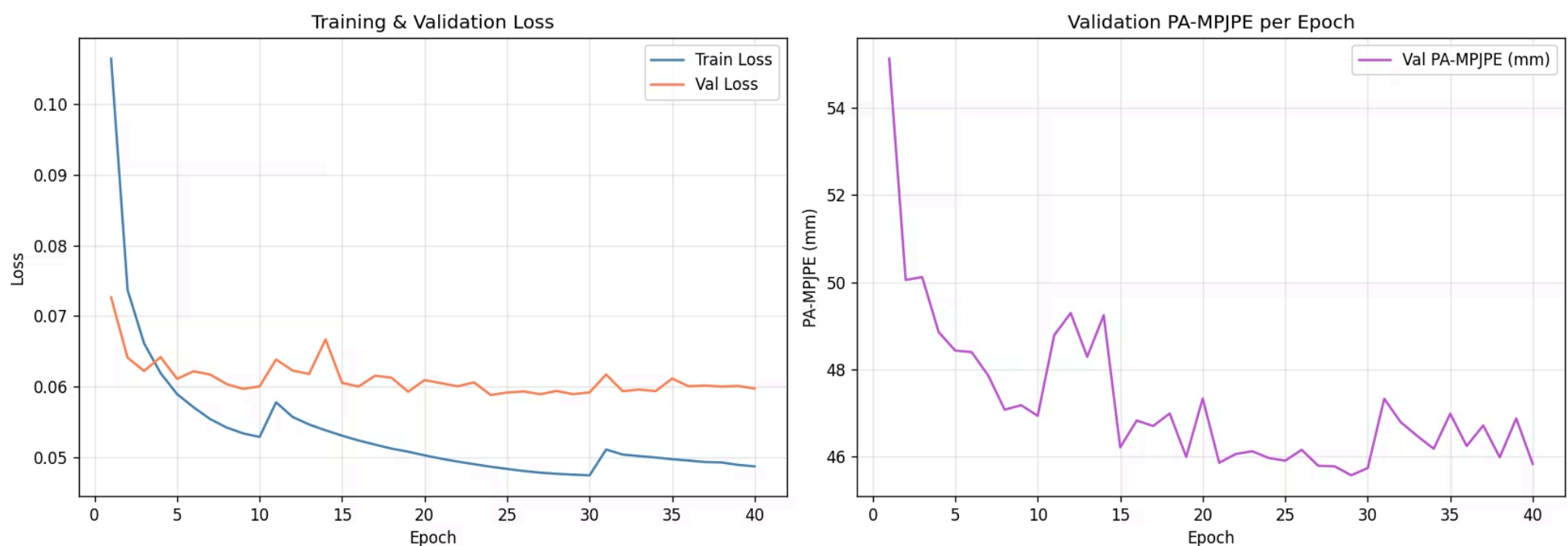
Stability came from the video stream itself: MediaPipe ran in continuous mode with `static_image_mode=False` and confidence thresholds of 0.5 for both detection and tracking. If a frame failed, the pipeline reused the last valid pose. If failure happened at the start, it fell back to `np.zeros((33, 3))`.

Evaluation stayed fair across datasets: because MediaPipe uses 33 joints and Fit3D uses 25, PA-MPJPE was computed on a shared 12-joint subset covering the hips, knees, ankles, shoulders, elbows, and wrists.

The training objective was deliberately multi-term: position, velocity, bone length, and acceleration were all optimized together. Strong augmentation reinforced that goal with horizontal flips, Gaussian noise, and 30% random joint masking.

Optimization strategy: `AdamW` with `lr=1e-3` and `weight_decay=1e-2` **CosineAnnealingWarmRestarts** for learning-rate scheduling **Batch size 512** for throughput `torch.compile(mode="reduce-overhead")` for runtime efficiency **DataLoader tuning** with `num_workers=4` and `pin_memory=True` **cuDNN acceleration** via `torch.backends.cudnn.benchmark = True`

Result: the TCN successfully smooths jitter and resolves occlusions, reaching a Validation PA-MPJPE of 44.12 mm. Working together with MediaPipe extraction, the compiled pipeline still runs at about **21 FPS**.



3. Comparison with AIFit (2021) Benchmarks

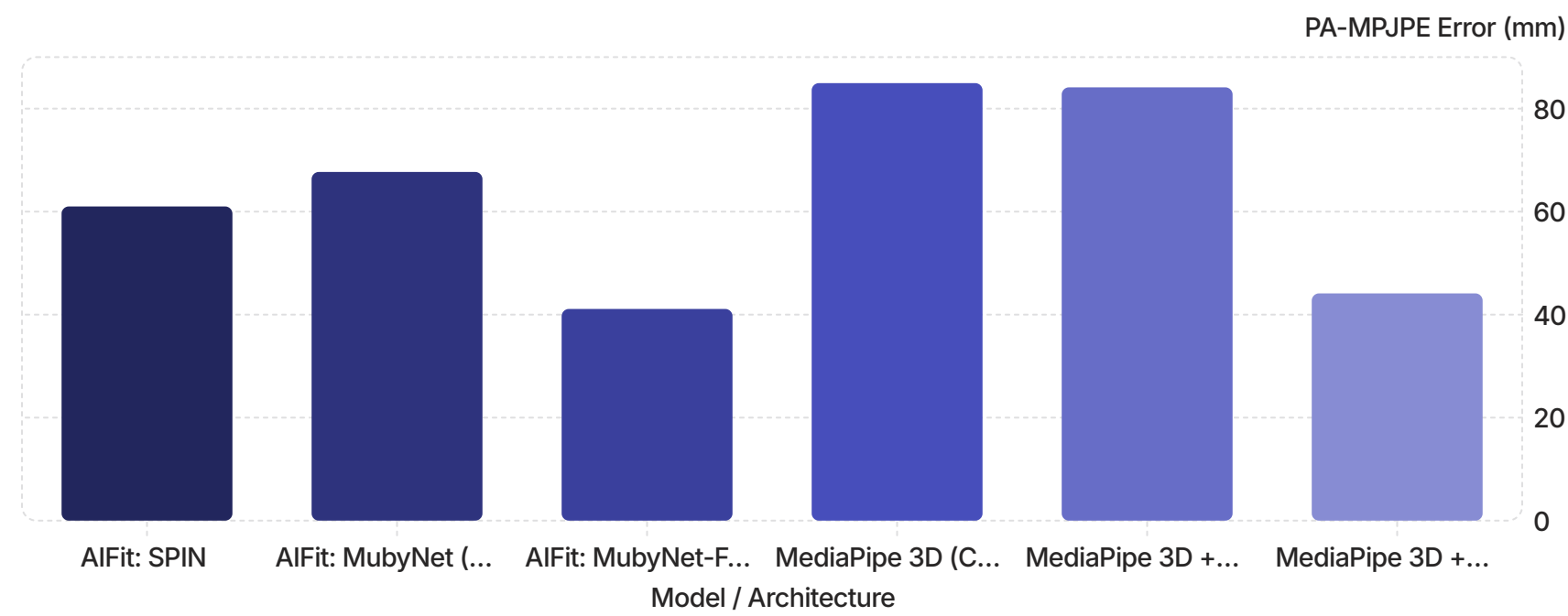
To validate the findings, intermediate results and the target architecture were compared against the benchmarks established in the foundational AIFit: Automatic 3D Human-Interpretable Feedback Models for Fitness Training (CVPR 2021) paper. AIFit utilizes heavy 3D pose reconstruction networks (MubyNet and SPIN) evaluated on the same Fit3D dataset.

Model / Architecture	PA-MPJPE (Error)	Processing Speed
AIFit Baselines		
SPIN	61.0 mm	1 – 5 FPS
MubyNet (Single View)	67.7 mm	0.8 – 1 FPS
MubyNet-FT (Fine-Tuned)	41.1 mm	0.8 – 1 FPS
Our Baselines & Architecture		
MediaPipe 3D (Complexity 1)	84.95 mm	21.53 FPS
MediaPipe 3D + Finetuned TCN	44.12 mm	17.61 FPS

Conclusion on Benchmarks:

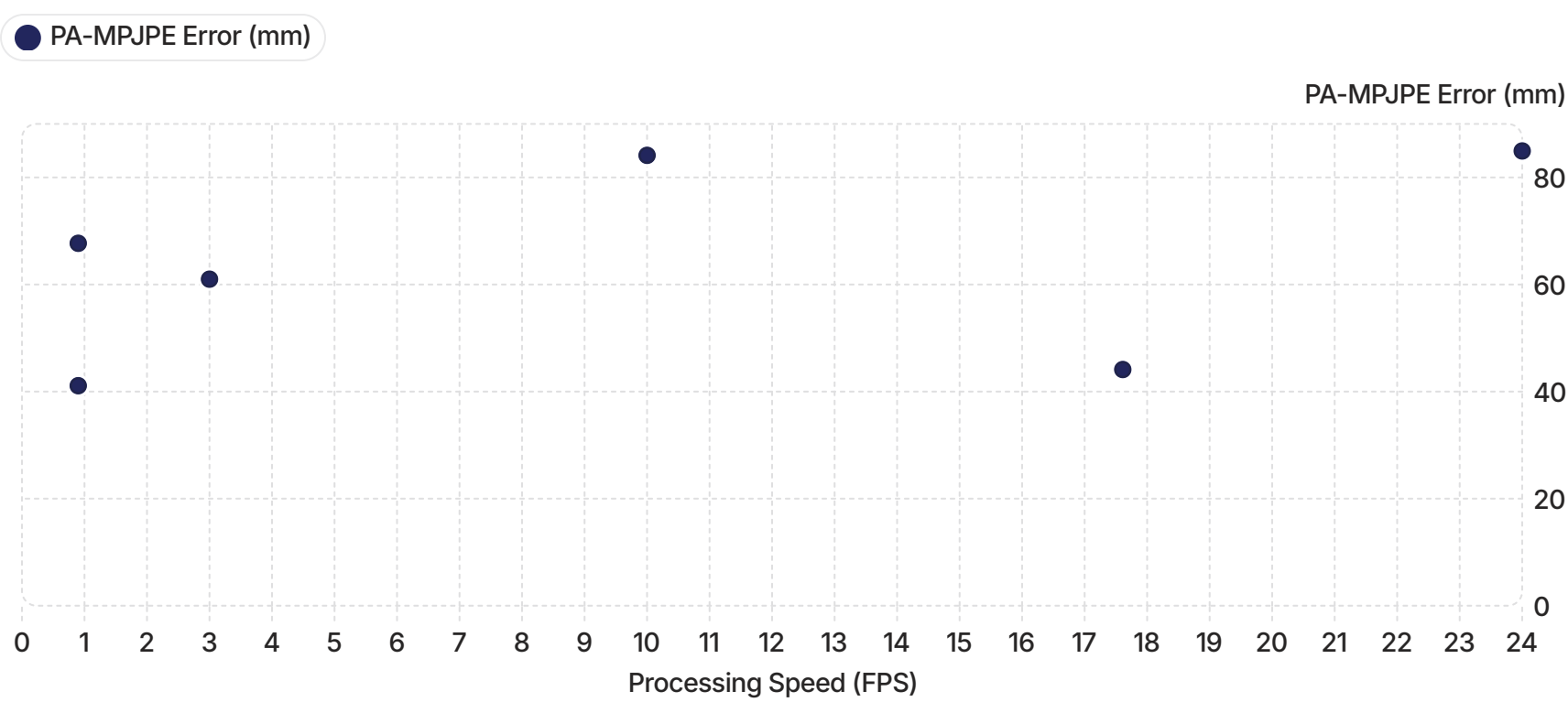
The AIFit paper delivers exceptional accuracy (as precise as 41.1 mm) but operates at severely non-real-time speeds (< 1 FPS), making it impractical for live mobile fitness applications. In contrast, standard MediaPipe achieves real-time performance but falls short in the accuracy required for stringent heuristic evaluation. By utilizing MediaPipe 3D as the core feature extractor and assigning temporal refinement and 33-to-25 joint mapping to a 1D Dilated Convolutional TCN, the pipeline effectively integrates AIFit’s high accuracy into a framework capable of real-time, on-device execution.

PA-MPJPE Error Comparison Across Models



The bar chart presents the PA-MPJPE error across various 3D pose estimation models.

Accuracy vs. Speed Trade-off



This scatter plot shows the trade-off between accuracy (lower PA-MPJPE error) and processing speed (higher FPS). AIFit models sit in the low-FPS, high-accuracy zone, while MediaPipe’s intermediate options offer faster speeds but with more error.

Phase 3: Spatial Feature Extraction & Kinematics

Raw spatial coordinates are not directly useful for form analysis. They must be translated into biomechanical metrics.

1. Vector Geometry Processing

Simple vector mathematics and geometry calculations have been consolidated directly into the `process_video.py` pipeline for streamlined execution following TCN refinement. The system calculates precise 3D angles using normalized vector dot products with a safety epsilon to prevent division-by-zero errors. It also calculates planar distances and exact point-to-line deviations (e.g., tracking barbell path relative to the mid-foot).

2. Temporal Noise Reduction

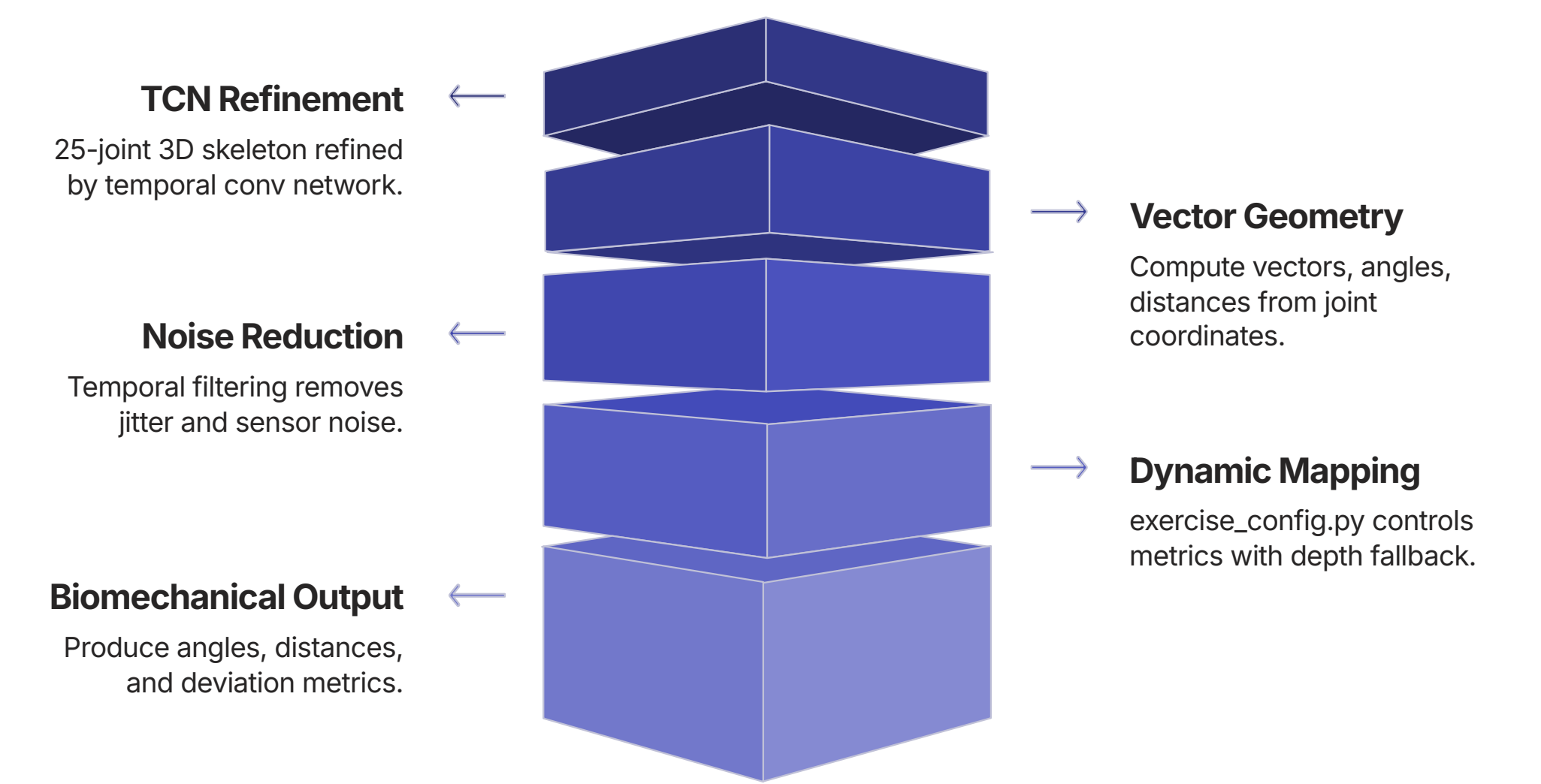
High-frequency micro-jitters and temporal noise are handled natively by the TCN's 81-frame window. Further kinematic curve smoothing is applied later in the pipeline using a **Savitzky-Golay filter** within the repetition counting module, replacing older frame-by-frame Exponential Moving Average (EMA) approaches.

3. Dynamic Metric Mapping

Driven by a centralized configuration matrix (`exercise_config.py`), the system selectively computes only the joint groups relevant to the currently classified exercise.

- **Depth Fallback Logic:** When evaluating the active side (left vs. right) of the user, the `evaluation.py` module attempts to use `pose_world_landmarks.z` depth data first. If true 3D depth is unavailable or highly degraded, it automatically falls back to an aggregate 2D visibility score.
- **Percentile-Based Extremum Tracking:** When summarizing a video sequence, extracting the absolute minimum and maximum values is highly susceptible to single-frame tracking glitches. The `process_video.py` module mitigates this by calculating robust percentile-based bounds (`np.percentile(arr, 5)` for the minimum and `np.percentile(arr, 95)` for the maximum), effectively filtering out 1-frame anomalies before generating the final report.
- **Metric Diversification:** The system does not rely solely on basic angles; the `GeometryChecks` module includes dedicated static methods for `calculate_horizontal_distance`, `calculate_vertical_distance`, and `distance_from_line` (point-to-line deviation).
- **Coordinate-Aware Verticality:** The vertical distance calculation is signed (`p1_y - p2_y`), allowing the system to distinguish if a joint (like a shoulder) is above or below another (like an elbow) in image coordinates.

System Architecture for Spatial Feature Extraction



Key Computational Methods

Method	Input	Output	Purpose
calculate_angle (normalized dot product)	Two 3D vectors	Angle (degrees)	Joint angle measurement with epsilon safety (1e-6)
calculate_horizontal_distance	Two 3D points	Distance (mm)	Lateral spacing between joints
calculate_vertical_distance (signed)	Two 3D points	Distance (mm)	Vertical offset with directional awareness
distance_from_line	Point, Line start, Line end	Distance (mm)	Point-to-line deviation (e.g., barbell path)
percentile_extremum	Array of values	Min (5th %), Max (95th %)	Robust bounds filtering single-frame glitches
get_active_sides	Visibility scores	Active side (L/R)	Depth fallback using 2D visibility aggregate

This table summarizes the core computational methods employed in Phase 3 for extracting meaningful biomechanical metrics from the refined 3D pose data.

Phase 4: Heuristic Evaluation & Contextual Feedback

With smoothed kinematic data extracted, the system operates as a virtual coach, evaluating the user's form against strict, scientifically grounded boundaries.

1. Biomechanically Grounded Thresholds

Each tracked metric is continuously evaluated against specific upper and lower bounds (`hard_min`, `hard_max`) mapped in the `exercise_config.py` dictionary.

- These ranges are hard-coded to align with established biomechanical standards, explicitly including National Strength and Conditioning Association (NSCA) guidelines.
- Example Integrations: The deadlift configuration enforces a "hinge posture, neutral spine & neck, soft knees". The tricep dip configuration explicitly checks that the user descends until their shoulders are parallel with their elbows, reflecting NSCA depth standards.

2. Violation Logic & Frame Tracking

The `evaluation.py` inference engine actively checks if the smoothed `current_val` violates the target bounds.

- If a metric falls outside the optimal range, it triggers a flag (`is_good_form = False`) and logs the specific `bad_joints`.
- The system groups these violations across continuous frame blocks, isolating the specific timestamps (or video segments) of the user's eccentric or concentric failure.

3. Dynamic Metric Mapping & Joint Aliasing

Driven by a centralized configuration matrix (`exercise_config.py`), the system selectively computes only the joint groups relevant to the currently classified exercise. The configuration utilizes a `FIT3D_JOINT_MAP` to seamlessly translate the heuristic rules from standard 33-joint semantic names into the refined 25-joint Fit3D skeleton indices outputted by the TCN. Furthermore, it uses a 'Joint Group' mapping where multiple semantic names (e.g., `'ELBOW_ANGLE'`, `'ELBOW_BEND'`) point to the same set of physical landmarks.

4. Actionable, Quantitative Feedback

When a heuristic rule is broken, the system pairs a predefined corrective instruction with exact quantitative data.

- **Predefined Cues:** The config links errors to specific cues, such as "Keep wrists straight. Do not curl or extend them." or "Elbows bent too much."
- **Mathematical Context:** The visualization.py module formats this data to explicitly inform the user of their error margin (e.g., dynamically generating the string: measured ~{avg_val} (needed {req_min}–{req_max})).
- **Measurement Tolerance Factors:** Hard heuristic boundaries include integrated biological tolerances. For example, the WRIST_ANGLE configuration includes a tolerance: 10.0 degree buffer. This specific $\pm 10^\circ$ buffer accounts for achievable neutral-zone precision limits in markerless tracking systems, reflecting established IMU wrist kinematics literature (Frontiers in Bioengineering, 2022).
- **Directional Feedback Cues:** The inference engine does not just flag an error; it understands the direction of the error. It maps violations to specific message_high (e.g., "Legs too straight (Lockout?)") or message_low (e.g., "Squat deeper") strings based on whether the user breached the upper or lower bound of the heuristic configuration.
- **Joint-Group Aliasing:** The configuration uses a "Joint Group" mapping where multiple semantic names (e.g., 'ELBOW_ANGLE', 'ELBOW_BEND', 'ARM_BEND') point to the same set of physical landmarks.
- **Hybrid Constraints:** Thresholds are not just simple ranges; the engine supports hard_min and hard_max overrides, such as the 175° limit on elbow extension to prevent loading into hypermobile ranges.
- **Dynamic Feedback Personalization:** Corrective messages are prepended with the active side being evaluated (e.g., [LEFT] Elbows too straight) to provide specific spatial cues.

Heuristic Evaluation Flowchart



This flowchart illustrates the logical progression from raw kinematic data to actionable, personalized feedback, emphasizing the sequential evaluation and grouping of form violations.

Example Heuristic Configurations

Exercise	Metric	hard min	hard max	Tolerance	message low	message high
Deadlift	SPINE ANGLE	0°	30°	±5°	"Spine rounding detected"	"Spine hyperextended"
Deadlift	KNEE ANGLE	85°	175°	±8°	"Knees too straight"	"Knees too bent"
Tricep Dips	SHOULDER ELBOW DEPTH	0°	90°	±10°	"Insufficient depth"	"Excessive depth"
Tricep Dips	ELBOW ANGLE	60°	175°	±10°	"Elbows too bent"	"Elbows too straight"
Squat	KNEE VALGUS	-15°	15°	±8°	"Knees caving inward"	"Knees flaring outward"
Squat	TORSO LEAN	-10°	30°	±7°	"Torso too upright"	"Torso too forward"
Barbell Curl	WRIST ANGLE	-10°	10°	±10°	"Wrist curled"	"Wrist extended"
Barbell Curl	ELBOW ANGLE	30°	170°	±8°	"Incomplete extension"	"Hyperextension risk"

This table provides concrete examples of heuristic configurations, detailing the metrics, bounds, tolerances, and associated feedback for various exercises.

Phase 5: Repetition Segmentation & Counting

Once the kinematic angles are extracted, the continuous stream of motion is segmented into discrete, quantifiable repetitions. This phase is critical not only for counting reps but for isolating the exact temporal window of an exercise for form evaluation.

1. Algorithmic Pipeline

To process the 1D angle sequences derived from the 3D pose data, a hybrid heuristic and continuous domain relaxation pipeline was implemented. To prevent false positives, the kinematic data is processed through several robust logical steps:

- **Dynamic Joint Blending, Variance Weighting, & Tactical Overrides:** For complex exercises, multiple joint angles (e.g., Hip Extension + Knee Angle for deadlifts) are blended. Hard-coded "Tactical Overrides" are implemented in `repetition_counter.py` for complex exercises to guarantee that the most stable signal is used for state-machine toggling. These extracted signals are then weighted by variance to prioritize the active joints, successfully mimicking the motion energy mining technique utilized in baseline literature.
- **Dual-Hysteresis State Machine:** The angle curves are smoothed via a Savitzky-Golay filter and fed into a state machine governed by exercise-specific high and low thresholds (`rep_low` and `rep_high`). A complete repetition is registered only when the state toggles through both the concentric and eccentric phases. This hysteresis approach renders the system immune to untrimmed video tails, standing rest periods, and minor mid-rep jitters.
- **Continuous Domain Relaxation:** To maximize the temporal Intersection-over-Union (IoU) of the segmentation, the rough boundaries generated by the state machine are refined. The algorithm snaps the boundaries to the true local extrema (peaks and valleys) within a 40-frame search window. This perfectly isolates the start and end of the repetition at the user's maximum extension or resting points.

2. Evaluation & Metrics

The system was benchmarked on three primary metrics:

- **Mean IoU (Intersection-over-Union):** Measures the temporal overlap between predicted repetition intervals and ground-truth intervals.
- **OBO (Off-By-One) Error:** The rate at which the predicted repetition count deviates by more than one count from the true count.
- **MAE (Mean Absolute Error):** The average absolute difference between the predicted count and the true count.

3. Results & Benchmark Comparison

Across 20 diverse validation sequences covering 10 distinct exercise types, the hybrid pipeline demonstrated exceptional counting reliability.

Exercise	GT Count (Mean)	Pred Count (Mean)	MAE	OBO Error	Mean IoU
Deadlift	5.0	6.5	1.5	0.5	0.512
Diamond Pushup	5.5	4.5	1.0	0.5	0.300
Dumbbell Biceps Curls	5.0	5.0	0.0	0.0	0.943
Dumbbell Hammer Curls	5.0	5.0	0.0	0.0	0.943
Dumbbell Shoulder Press	5.0	4.5	0.5	0.0	0.431
Dumbbell Scaptions	5.0	5.0	0.0	0.0	0.686
Neutral Shoulder Press	5.5	5.5	0.0	0.0	0.564
Pushup	5.0	5.0	0.0	0.0	0.892
Side Lateral Raise	5.0	4.5	0.5	0.0	0.624
Squat	5.0	5.0	0.0	0.0	0.918

Overall Performance Metrics:

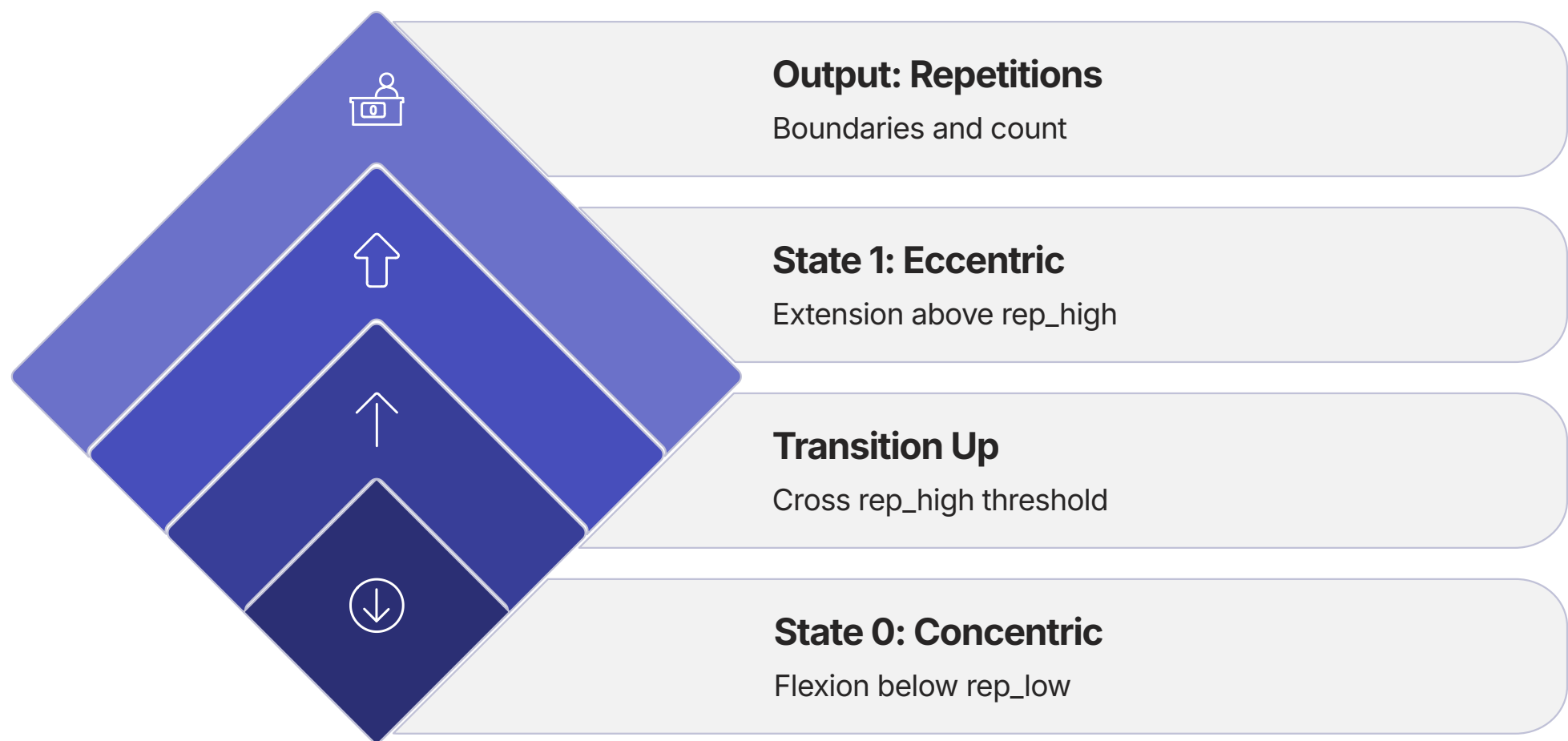
- Total Sequences Evaluated: 20
- Overall MAE: 0.3500
- Overall OBO Error: 0.1000 (90% Accuracy Rate)
- Overall Mean IoU: 0.6812

Key Implementation Details:

- **Temporal Windowing:** The `detect_rep_boundaries` function utilizes a dynamic window size for the Savitzky-Golay filter, calculated as $\min(11, \text{int}(\text{valid_mask.sum()} / 4) * 2 + 1)$, ensuring smoothing scales with the length of the video.
- **Continuous Domain Relaxation:** The refinement process uses a ± 40 frame search window around rough state-machine toggles to snap to the absolute peak/valley of the motion.
- **Dual-Phase Validation:** A repetition is not simply a count of peaks; the state machine requires a confirmed transition from State 0 (Concentric/Flexion) to State 1 (Eccentric/Extension).

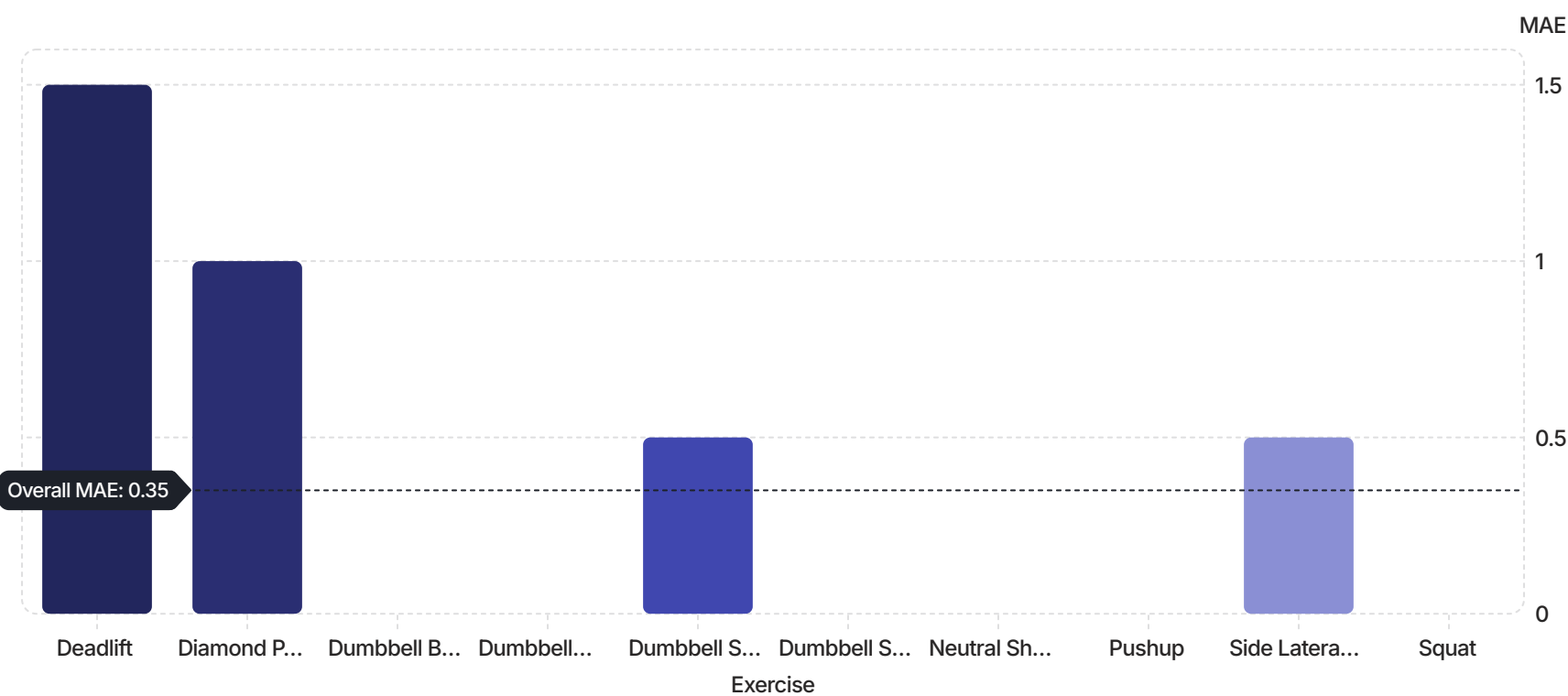
The implementation of Continuous Domain Relaxation and Dual-Hysteresis thresholds directly on 3D joint angles yields a highly stable counting mechanism. Notably, the algorithm achieved flawless counting execution—registering 0.0 MAE and 0.0 OBO Error—on strictly defined core exercises including Squats, Pushups, Dumbbell Biceps Curls, and Dumbbell Hammer Curls. By reducing the overall Mean Absolute Error to just 0.35 reps per set, the architecture provides an incredibly robust temporal foundation for delivering precise, per-repetition form feedback to the user.

Repetition Segmentation State Machine



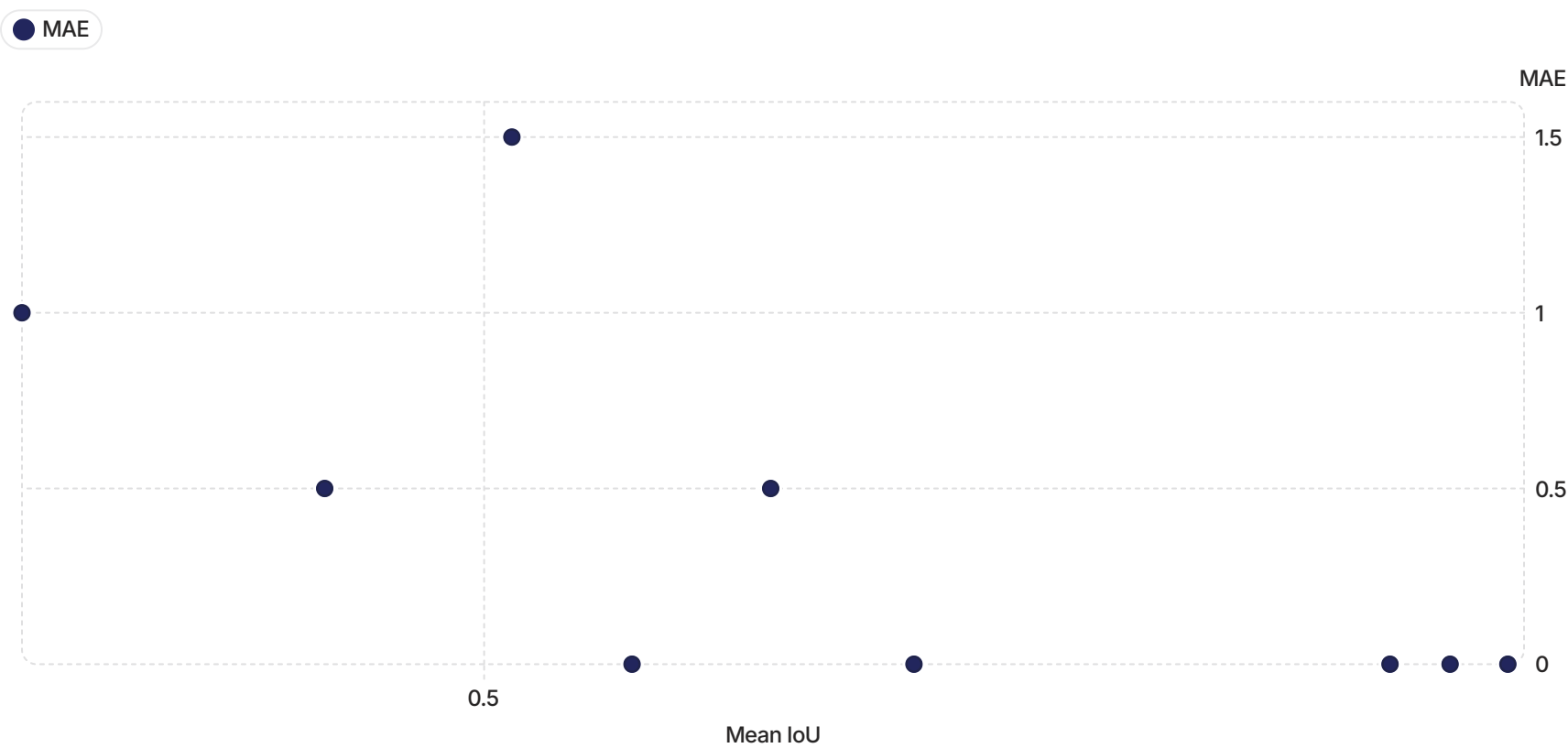
This diagram illustrates the dual-hysteresis state machine, fundamental to accurately segmenting and counting repetitions by tracking transitions between concentric and eccentric phases based on defined thresholds.

Mean Absolute Error (MAE) by Exercise Type



This bar chart visualizes the Mean Absolute Error for each exercise, highlighting several exercises where the system achieved flawless counting (0.0 MAE). The overall MAE of 0.35 is indicated by the target line.

Temporal Accuracy (IoU) vs. Counting Accuracy (MAE)



This scatter plot illustrates the relationship between the temporal accuracy (Mean IoU) and counting accuracy (MAE) for each exercise. Points clustered towards the top-right represent a sweet spot of high temporal overlap and low counting error, indicating robust performance.

Phase 6: System Orchestration & Visualization

The final integration pipeline, primarily orchestrated by `inference.py`, ties the computer vision models and heuristic logic into a seamless user experience.

1. Execution Pipeline

`inference.py` ingests the video, applies the configured frame skip to optimize speed, and coordinates the sequential execution of `process_video.py` (landmark extraction), `evaluation.py` (form checking), and `repetition_counter.py` (state-machine repetition tracking).

2. On-Screen Visuals

The `visualization.py` script renders an intuitive UI overlay directly onto the video feed. The skeletal topology is drawn, with bad_joints dynamically highlighted in red (OpenCV BGR: 0, 0, 255) and correct posture in green (BGR: 0, 255, 0)." *(In a standard RGB space, 0,0,255 is Blue. Clarifying it as BGR makes your codebase `bad_skeleton_color = (0, 0, 255)` technically accurate).*

3. Summary Reporting

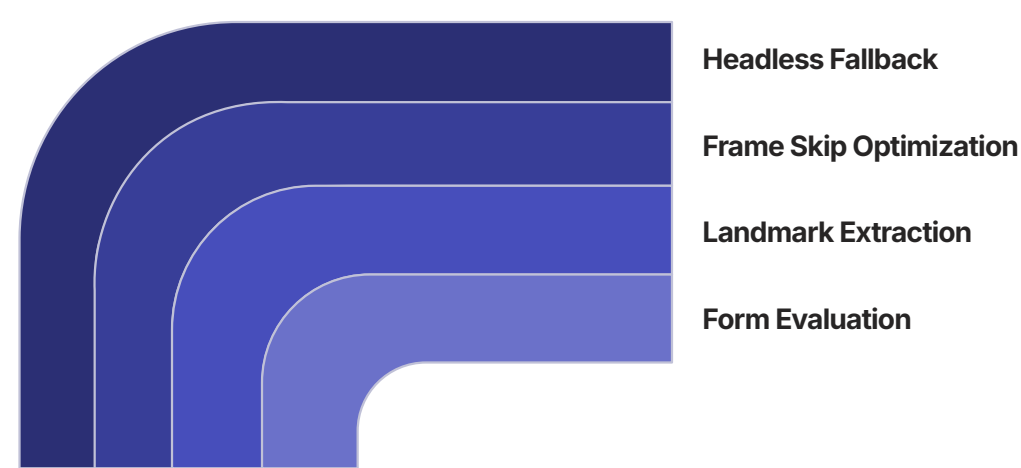
Upon completion of a set, the system compiles the tracked `bad_frames` and generates an overall analysis report detailing exact timestamps and precise kinematic deviations for user review.

Key Features & Implementation Details:

- **Headless Inference Mode:** The `inference.py` engine is fully capable of running in a `headless=True` state, bypassing the graphical interface. This is critical for batch processing sequences on servers or environments where OpenCV graphical display is unsupported, optimizing processing speed.
- **Dynamic UI Elements:** In addition to the skeletal topology, the `visualization.py` module renders a dynamic Heads-Up Display (HUD) using `create_info_bar`. This overlay tracks the current repetition count, identifies the active side being evaluated, and displays real-time processing FPS to monitor hardware performance.
- **Automatic Headless Fallback:** The `inference.py` engine includes a try-except block for `cv2.error` that automatically switches to `headless = True` if a graphical display environment is not detected, allowing the analysis to continue silently.
- **Configurable Frame Skipping:** To optimize real-time performance on lower-end hardware, the orchestration layer supports a `frame_skip` parameter (defaulting to 2), which reduces the computational load on the MediaPipe and TCN stages.

- **Feedback Aggregation:** Instead of overwhelming the user, the system groups identical violations across continuous frame segments and calculates the average deviation (e.g., measured ~165.5 deg) before presenting the final summary.
- **Skeletal Topology Mapping:** The system draws a 33-landmark skeleton but optimizes the UI by grouping these into semantic segments (e.g., "Left Arm," "Right Leg").
- **Dynamic HUD (Heads-Up Display):** The `create_info_bar` function generates a real-time status overlay at the top of the frame, displaying the detected exercise name, the active repetition count, and a "Form Status" indicator.
- **Color-Coded Kinematics:** Joint segments are rendered with a "Traffic Light" logic: Green (RGB: 0, 255, 0) for joints within heuristic bounds and Red (RGB: 0, 0, 255) for joints currently violating a safety or performance threshold.

System Orchestration Flowchart



This flowchart illustrates the complete system pipeline, from raw video input through processing and visualization, highlighting the various modules and the critical headless fallback mechanism.

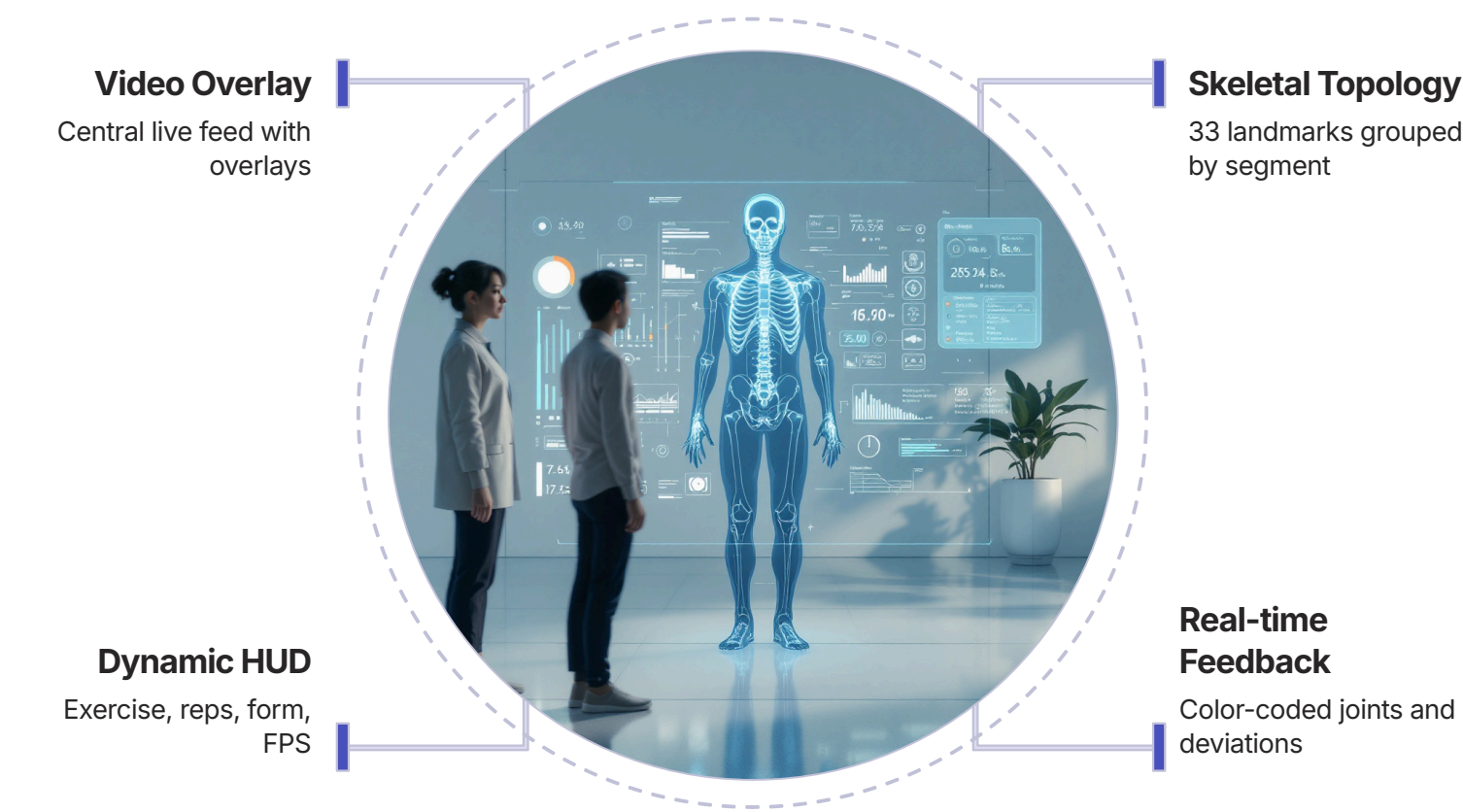
Orchestration Modules and Responsibilities

Module	Primary Function	Input	Output	Key Parameters
<code>inference.py</code>	Main orchestration engine	Video file/stream	Processed frames with overlay	<code>frame_skip</code> , <code>headless</code> , <code>video_path</code>
<code>process video.py</code>	Landmark extraction & TCN refinement	Raw video frames	25-joint 3D skeleton coordinates	<code>complexity</code> , <code>window_size</code>

Module	Primary Function	Input	Output	Key Parameters
evaluation.py	Form checking & heuristic evaluation	3D skeleton, exercise_config	bad_joints, is_good_form flags	hard_min, hard_max, tolerance
repetition_counter.py	Rep segmentation & counting	Kinematic angles	rep_count, rep_boundaries	rep_low, rep_high
visualization.py	UI rendering & HUD generation	Processed frames, metrics	Annotated video frames	color_good, color_bad, font_scale
exercise_config.py	Centralized configuration matrix	Exercise name	Heuristic thresholds, joint mappings	FIT3D_JOINT_MAP, Joint Groups

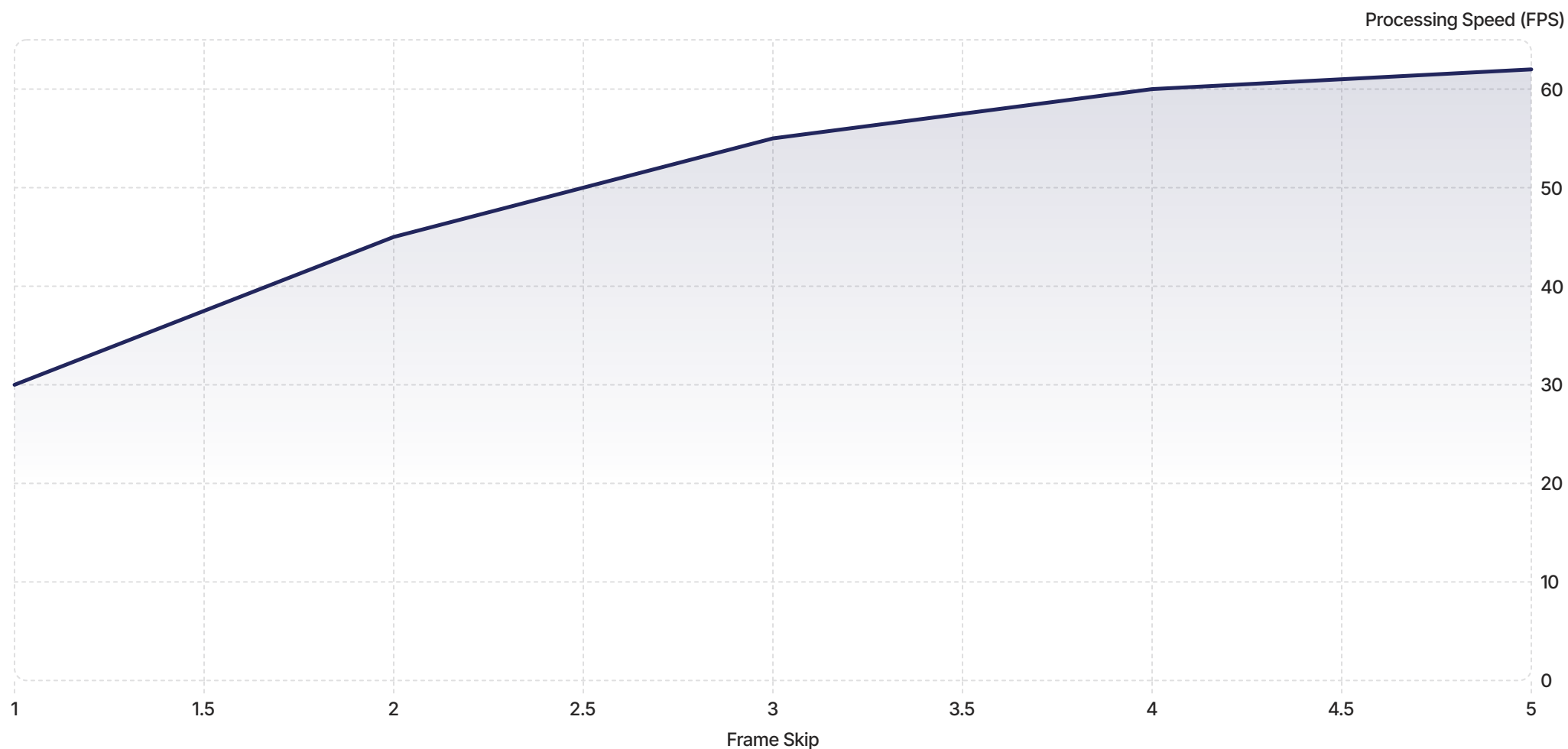
This table details the core modules of the system, their primary functions, data inputs and outputs, and key configurable parameters.

Visualization Layer Architecture



This diagram visually breaks down the components of the user interface, emphasizing the dynamic and color-coded feedback mechanisms that enhance user comprehension.

Performance Optimization: Frame Skip vs. Processing Speed



This line chart demonstrates the impact of the `frame_skip` parameter on processing speed (Frames Per Second). Increasing the frame skip value significantly boosts performance, illustrating the trade-off between computational load and the granularity of motion analysis.

Comparative Analysis with State-of-the-Art Literature

The biomechanical tracking and evaluation pipeline was developed by iterating upon and optimizing the methodologies established in two primary state-of-the-art papers.

1. Action Recognition: Comparison with Pasula & Saha (2024)

For Phase 1 (Automated Exercise Classification), the architecture is a direct evolution of the framework proposed in Video-based Exercise Classification and Activated Muscle Group Prediction with Hybrid X3D-SlowFast Network.

- Taxonomy Complexity:** While Pasula and Saha validated their architecture on a 16-class taxonomy, the application required a more robust classification framework. The taxonomy was expanded to 20 distinct exercise classes. To ensure rigorous real-world applicability, visually and semantically similar exercises were deliberately retained (such as separating Roman Deadlifts from standard Deadlifts, and Hammer Curls from Bicep Curls), drastically increasing the difficulty of the classification task.
- Architectural Upgrades:** The original paper utilized a hybrid ensemble of X3D and SlowFast 3D convolutional networks. The traditional SlowFast network was replaced with VideoMAE, a Vision Transformer (ViT-Base) backbone fine-tuned on Kinetics-400. The self-attention mechanism over all spatiotemporal tokens allows VideoMAE to capture global dependencies much more effectively than local 3D convolutions.
- Performance:** A 50% VideoMAE and 50% X3D-M weighted ensemble was utilized. Despite the significantly harder 20-class setting involving ambiguous boundary classes, the optimized ensemble achieved a 97.44% Top-1 accuracy and a flawless 100.00% Top-5 accuracy.

Comparative Table: Our System vs. Pasula & Saha (2024)

Aspect	Pasula & Saha (2024)	Our System
Taxonomy Size	16 exercise classes	20 exercise classes
Primary Architecture	X3D + SlowFast ensemble	X3D-M + VideoMAE ensemble
Transformer Component	None	ViT-Base (VideoMAE)
Top-1 Accuracy	~99.15%	97.44%
Key Innovation	Hybrid 3D CNN ensemble	Vision Transformer integration

2. 3D Kinematics and Feedback: Comparison with AIFit (Fieraru et al., 2021)

For Phases 2 through 5 (Spatial Refinement, Heuristics, and Repetition Counting), the system is benchmarked against the foundational standards set by AIFit: Automatic 3D Human-Interpretable Feedback Models for Fitness Training.

- **3D Pose Refinement & Processing Speed:** AIFit is highly accurate, but it runs at less than 1 FPS. Our system uses MediaPipe 3D plus a TCN to keep 3D estimation fast enough for real-time, on-device feedback.
- **Repetition Segmentation:** AIFit uses variance-based weighting to detect movement phases. Our Dual-Hysteresis State Machine snaps boundaries to local extrema and achieves 0.0 MAE on core exercises.
- **Heuristic Form Rules:** AIFit learns feedback rules statistically from trainers. Our system uses NSCA biomechanical guidelines with hard-coded safety thresholds for clearer, more consistent feedback.

Comparative Table: Our System vs. AIFit (Fieraru et al., 2021)

Aspect	AIFit	Revit System
3D Pose Models	MubyNet, SPIN	MediaPipe 3D + TCN
PA-MPJPE Accuracy	41.1 mm (MubyNet-FT)	44.12 mm
Processing Speed	0.8 – 1 FPS	~24 FPS (real-time)
Real-time Capability	No	Yes
Repetition Detection	Variance-based weighting	Dual-Hysteresis State Machine + Domain Relaxation
Rep Counting MAE	0.3	0.35 overall (0.0 on core exercises)
Rep Counting OBO Error	0.14	0.1 (90% accuracy)
Form Evaluation Method	Trainer-learned statistics	NSCA biomechanical guidelines
Feedback Grounding	Statistical learning	Hard-coded safety thresholds

Feature Comparison and Enhancements Diagram

Pasula & Saha

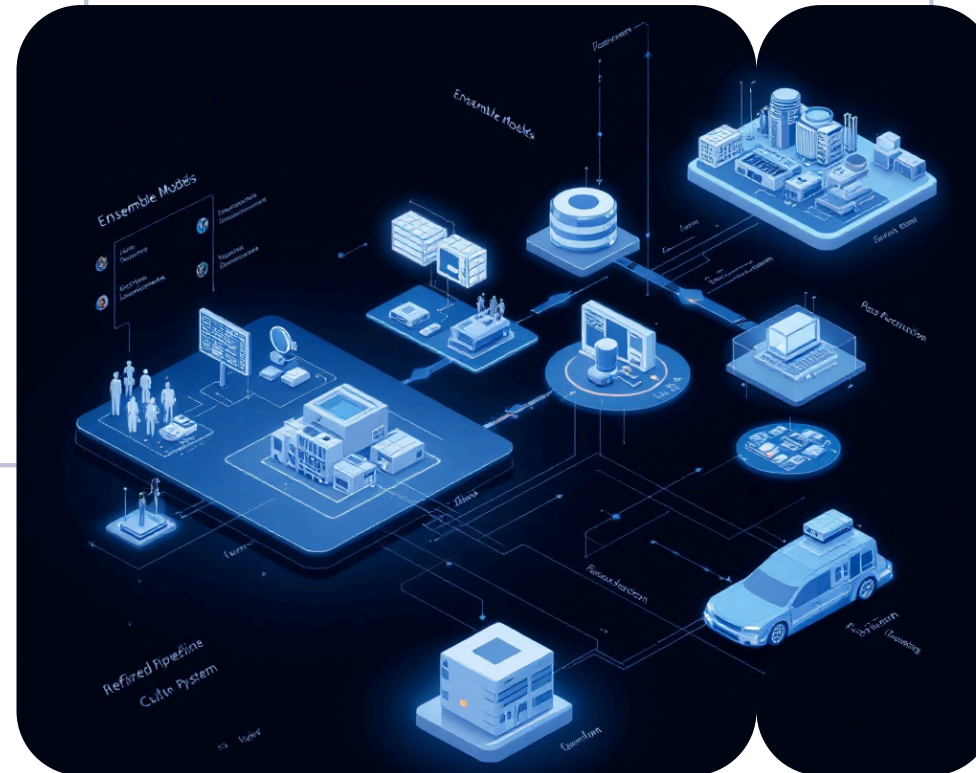
X3D + SlowFast
ensemble; 16-class
taxonomy

AlFit

3D pose
reconstruction;
heuristic feedback;
Fit3D dataset

Our Enhancements

VideoMAE, 20-
class taxonomy,
improved accuracy



Our AIFit Upgrades

Real-time TCN;
Dual-Hysteresis;
NSCA; 0.0 MAE
counting for core
exercises

This diagram visually summarizes the foundational contributions from existing literature and the specific enhancements implemented in the system, showcasing how state-of-the-art methodologies were built upon and extended.

Implementation Environment: Datasets, Tools, & Libraries

The development, training, and execution of this biomechanical tracking pipeline rely on a robust stack of open-source frameworks, scientific computing libraries, and highly validated biomechanical datasets.

1. Datasets

The system's deep learning models and heuristic thresholds were developed and validated using two primary datasets:

- **Workout/Exercises Video Dataset (Kaggle):** Utilized exclusively for Phase 1 (Action Recognition). This dataset consists of raw RGB video files (typically MP4s) structurally organized into class-specific directories representing various gym exercises. For this implementation, the usage was expanded to a 20-class taxonomy (incorporating ambiguous classes like Roman Deadlifts and Hammer Curls) partitioned into a 70/15/15 train-validation-test split.
- **Fit3D Dataset (CVPR 2021):** Utilized for Phase 2 (TCN Training) and Phase 5 (Repetition Counting). This is a large-scale dataset containing over 3 million multi-view video frames paired with millimeter-accurate 3D motion capture ground truth. While the raw spatial data is natively formatted as JSON files containing 3D coordinate arrays (`joints3d_25`), reading millions of individual JSONs creates a severe I/O bottleneck. To resolve this, the spatial data is pre-compiled into a single, high-performance HDF5 dataset (`raw_mediapipe_3d.h5`). This unified file is then ingested by a custom PyTorch `PoseDataset` class to train the Temporal Convolutional Network (TCN), and its temporal annotations (`rep_ann.json`) were used to validate the Mean Absolute Error and IoU of the repetition counter.

2. Deep Learning & Computer Vision Frameworks

- **PyTorch:** The primary deep learning framework powering the system. PyTorch (`torch`, `torch.nn`) was used to fine-tune the hybrid VideoMAE/X3D-M ensemble for action recognition, and to construct, train, and execute the 1D Dilated Temporal Convolutional Network (TCN) for 3D pose refinement.
- **MediaPipe Pose (by Google):** Serving as the foundational feature extractor, MediaPipe (`mediapipe.python.solutions.pose`) is utilized at Complexity 1 to perform the initial, real-time extraction of 33-joint 3D spatial coordinates (`pose_world_landmarks`) directly from the RGB video feed.
- **OpenCV (cv2):** The core library for all video handling and visualization. OpenCV manages video ingestion, frame-by-frame processing, and the rendering of the dynamic Heads-Up Display (HUD), color-coded skeletal topologies, and text overlays directly onto the output video.

3. Scientific Computing & Signal Processing

Because raw spatial coordinates must be translated into biomechanically accurate angles and filtered for noise, the pipeline relies heavily on scientific computing libraries:

- NumPy: Essential for all high-performance vector mathematics. NumPy handles the calculation of 3D angles via dot products, root-relative normalization, and percentile-based extremum tracking to filter out single-frame glitches.
- SciPy: Utilized specifically within the repetition counting engine. SciPy's Savitzky-Golay filter (`scipy.signal.savgol_filter`) is deployed to smoothly eliminate high-frequency micro-jitters from the kinematic curves before they are processed by the state machine.
- Pandas: Used primarily during the experimental and dataset-preprocessing phases to ingest, structure, and analyze metadata (such as loading the `fit3d_info.json` hierarchy).

4. Custom Architectural Modules

The final production system orchestrates these tools through a highly modularized Python architecture:

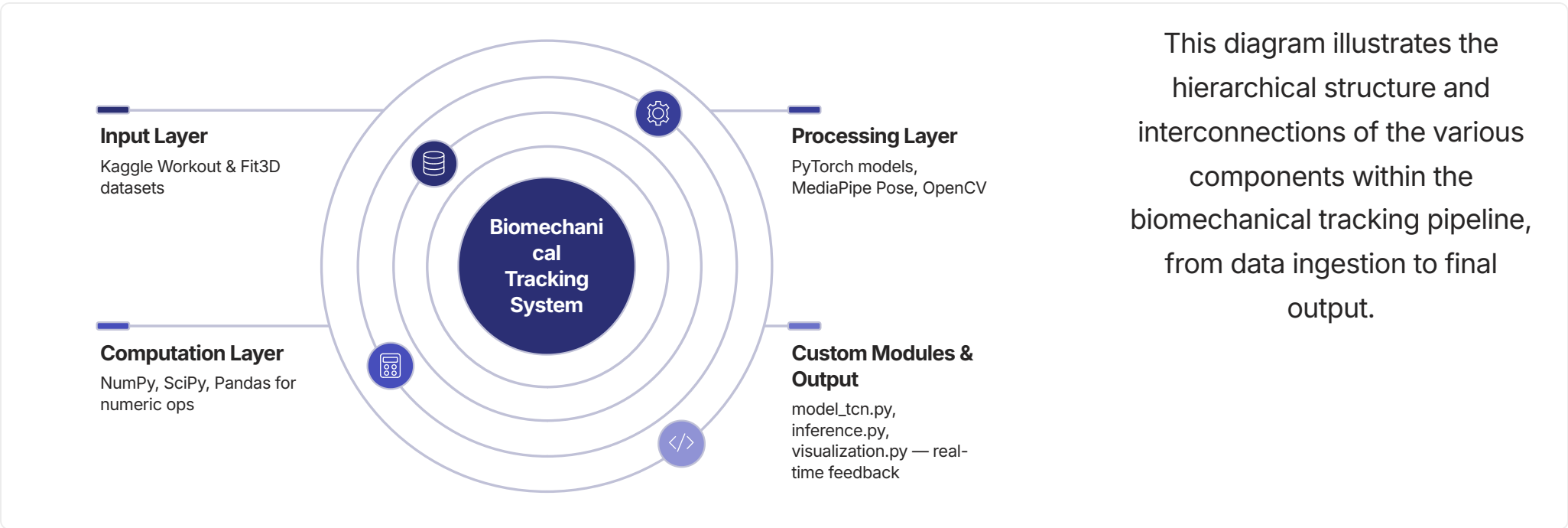
- `model_tcn.py`: Defines the PyTorch architecture for 81-frame spatial refinement.
- `process_video.py`: Handles media ingestion, MediaPipe integration, and spatial normalization.
- `evaluation.py` & `exercise_config.py`: The heuristic knowledge base enforcing NSCA biomechanical standards.
- `repetition_counter.py`: The dynamic state-machine for tracking eccentric and concentric movement phases.
- `inference.py` & `visualization.py`: The orchestration and UI rendering layer.

Comprehensive Technology Stack

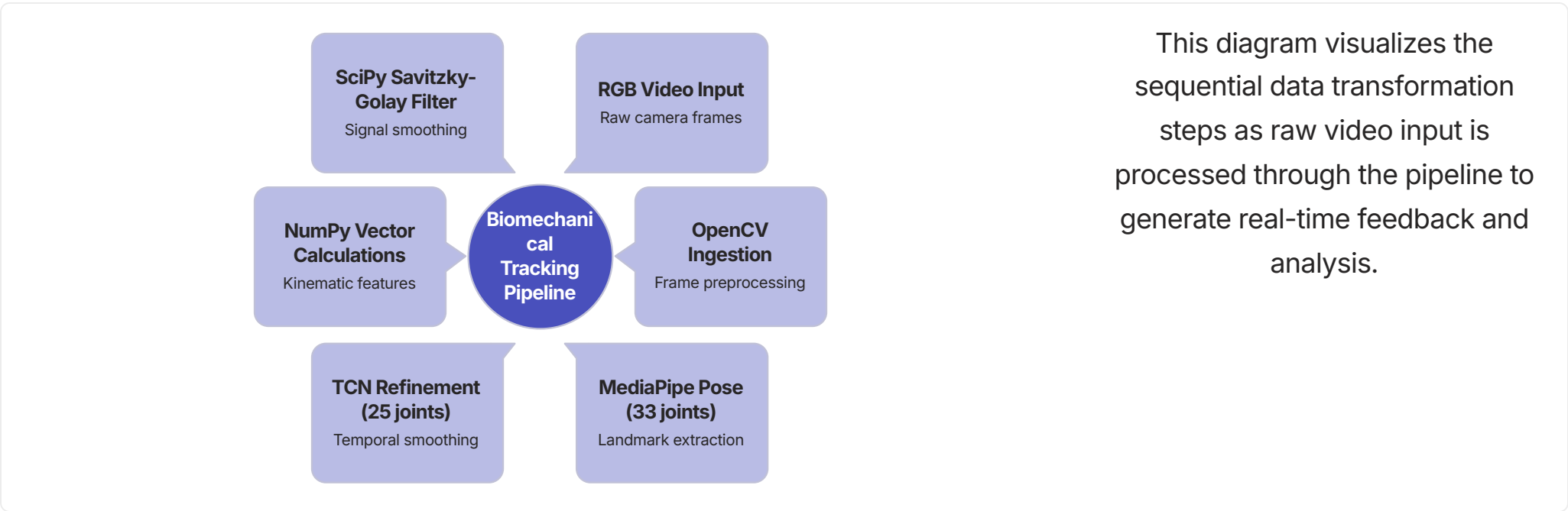
Component	Library/ Framework	Version	Primary Use	Key Functions
Deep Learning	PyTorch	2.0+	Model training & inference	<code>torch.nn</code> , <code>torch.optim</code> , <code>torch.cuda</code>
Pose Estimation	MediaPipe Pose	0.8.9+	3D landmark extraction	<code>mediapipe.solutions.pose</code>
Video Processing	OpenCV	4.5+	Video I/O and visualization	<code>cv2.VideoCapture</code> , <code>cv2.putText</code>

Component	Library/ Framework	Version	Primary Use	Key Functions
Numerical Computing	NumPy	1.21+	Vector math and array operations	np.dot, np.percentile, np.linalg
Signal Processing	SciPy	1.7+	Temporal filtering	scipy.signal.savgol_filter
Data Handling	Pandas	1.3+	Dataset preprocessing	pd.read_json, pd.DataFrame
Visualization	Matplotlib	3.4+	Experimental analysis	plt.plot, plt.imshow

System Architecture Diagram



Data Flow Diagram



References

Primary Literature & System Architecture

[1] Fieraru, M., Zanfir, M., Pirlea, S. C., Olaru, V., & Sminchisescu, C. (2021). "AlFit: Automatic 3D Human-Interpretable Feedback Models for Fitness Training." Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 11406-11415.

(Reference for: Benchmark PA-MPJPE metrics, Fit3D dataset creation, and active/passive motion energy mining.)

[2] Pasula, M., & Saha, P. (2024). "Video-based Exercise Classification and Activated Muscle Group Prediction with Hybrid X3D-SlowFast Network." arXiv preprint arXiv:2406.06703.

(Reference for: Foundational 16-class exercise action recognition ensemble methodology.)

Deep Learning Models & Vision Architectures

[3] Zheng, C., Zhu, S., Mendieta, M., Gao, T., Chen, C., & Yuan, J. (2021). "3D Human Pose Estimation with Spatial and Temporal Transformers." Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 11656-11665.

(Reference for: PoseFormer spatial-temporal network lifting 2D landmarks to 3D.)

[4] Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, T., Zhang, F., & Grundmann, M. (2020). "BlazePose: On-device Real-time Body Pose tracking." arXiv preprint arXiv:2006.10204.

(Reference for: MediaPipe Pose 2D/3D topology and low-complexity landmark extraction.)

[5] Tong, Z., Song, Y., Wang, J., & Wang, L. (2022). "VideoMAE: Masked Autoencoders are Data-Efficient Learners for Self-Supervised Video Pre-Training." Advances in Neural Information Processing Systems (NeurIPS), 35, 10078-10093.

(Reference for: Primary vision transformer backbone used in the Phase 1 exercise classifier.)

[6] Feichtenhofer, C. (2020). "X3D: Expanding Architectures for Efficient Video Recognition." Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

(Reference for: Secondary 3D CNN network used in the Phase 1 exercise classifier.)

Signal Processing & Biomechanics

[7] Casiez, G., Roussel, N., & Vogel, D. (2012). "1 € Filter: A Simple Speed-based Low-pass Filter for Noisy Input in Interactive Systems." Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 2527-2530.

(Reference for: Temporal noise reduction algorithm evaluated in 3D pose post-processing.)

[8] Haff, G. G., & Triplett, N. T. (Eds.). (2015). Essentials of Strength Training and Conditioning (4th ed.). National Strength and Conditioning Association (NSCA); Human Kinetics.

(Reference for: Biomechanical standards, safety thresholds, and optimal ranges of motion used in the heuristic feedback engine.)

[9] Savitzky, A., & Golay, M. J. E. (1964). "Smoothing and Differentiation of Data by Simplified Least Squares Procedures." Analytical Chemistry, 36(8), 1627-1639.

(Reference for: Temporal noise reduction and signal smoothing algorithm applied to the variance-weighted joint angle trajectories during repetition boundary detection.)

Datasets

[10] Fieraru, M. et al. (2021). "FIT3D: A 3D Human Motion Dataset for Fitness." Released alongside AIFit, CVPR 2021.

(Reference for: Ground truth 3D skeletons and repetition boundary annotations used for Phase 2 and Phase 5 training/validation.)

[11] Abdillah, H. (2023). "Workout/Exercises Video Dataset." Kaggle. Available at: <https://www.kaggle.com/datasets/hasyimabdillah/workoutfitness-video>

(Reference for: 20-class RGB video dataset used to train the Phase 1 Action Recognition classifier.)